# Gestión para la generación automática de menús nutricionales


INTEGRANTES

| | |
|---|---|
| Enrique Alejandro Gonzalez Moreira | C312 |
| Javier Fontes Basabe | C312 |
| Heily Rodriguez Rodriguez | C312 |
| Ernesto Alejandro Soler Choong | C312 |
| Mauricio Bridon Carbo | C312 |

**Team-Bales**


Bases de Datos II · Ingeniería de Software

*Quinto Semestre · Ciencias de la Computación · Curso 2026–2027*

# Índice

**1. Introducción** . . . . . **1**

**2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema** . . . . . **2**

2.1 Requerimientos funcionales . . . . . 2

2.2 Requerimientos informacionales . . . . . 6

**3. MERX de la Base de Datos** . . . . . **11**

**4. Especificación de las restricciones de integridad establecidas y/o detectadas** . . . . **14**

4.1 Restricciones estructurales . . . . . 14

4.2 Restricciones semánticas . . . . . 18

4.3 Restricciones de la ficha nutricional (MongoDB) . . . . . 19

4.4 Restricciones definidas en la consolidación . . . . . 19

4.5 Lo que deliberadamente no se restringe . . . . . 22

**5. Valoración de la corrección del diseño de la base de datos** . . . . . **23**

5.1 Conjunto de dependencias funcionales extraído del MERX . . . . . 23

5.2 Cubrimiento minimal . . . . . 26

5.3 Descomposición por el algoritmo de síntesis . . . . . 30

5.4 Propiedades de la descomposición . . . . . 34

5.5 Valoración final del diseño . . . . . 36

5.6 Flexibilidad y extensibilidad . . . . . 38

**6. Conclusiones** . . . . . **39**

# 1. Introducción

El presente informe describe el diseño de la base de datos para el sistema de gestión de menús nutricionales en un entorno hospitalario. El sistema atiende a los actores de la §2.1: pacientes que reciben y consumen menús, nutricionistas que los crean y administran el banco de alimentos y platos, un jefe de nutrición que supervisa y valida los contenidos (el rol de revisor), y administradores que mantienen el sistema. Su función central es automatizar la generación de menús a partir de dietas definidas institucionalmente, garantizar su validación clínica y registrar el consumo y los resultados nutricionales de cada paciente.

El trabajo sigue una metodología progresiva de cuatro etapas. En primer lugar, se analizan y reformulan los requerimientos funcionales e informacionales para precisar las operaciones que el sistema debe soportar y las consultas que debe responder. A partir de ese análisis se construye el Modelo Entidad-Relación Extendido (MERX) consolidado, que integra las entidades, atributos, relaciones y agregaciones del dominio en un único diagrama coherente. Sobre ese modelo se especifican las restricciones de integridad del esquema, clasificadas por su origen y declaradas mediante los mecanismos estándar del motor de base de datos. Finalmente, se verifica la corrección formal del diseño comprobando que cada esquema de la descomposición alcanza al menos la

Tercera Forma Normal (3FN) y que la descomposición satisface la Propiedad de Preservación de Dependencias Funcionales (PPDF) y la Propiedad del Join sin Pérdida de Información (PLJ). La Forma Normal de Boyce-Codd (BCNF) se comprueba después, como verificación adicional.

# 2. Análisis y reformulación enriquecedora de los requerimientos funcionales e informacionales del sistema

## 2.1 Requerimientos funcionales

### Descripción del dominio

El sistema gestiona la **generación automática de menús nutricionales** dentro de una institución de salud. Su núcleo es un banco de alimentos y platos que los nutricionistas administran, clasifican y usan para confeccionar menús adaptados a las dietas de los pacientes hospitalizados. El menú puede generarse algorítmicamente — aplicando parámetros de proporción por tipo de preparación, cobertura de grupos nutricionales y cantidad total de alimentos — o confeccionarse de forma manual bajo criterios del jefe de nutrición; en cualquier caso queda sujeto a validación antes de ser asignado a un paciente.

### Actores identificados

| Actor | Rol en el sistema |
|---|---|
| **Nutricionista** | Ingresa y clasifica alimentos y platos en el banco; crea menús; ejecuta revalorizaciones. Actor central del sistema. |
| **Jefe de nutrición** | Rol de Nutricionista con potestad de revisar y aprobar los menús generados, o indicar la confección de uno nuevo bajo sus criterios. |
| **Revisor** | Rol de Nutricionista que actúa en el proceso de validación de un menú: registra fecha, observaciones y dictamen (aprobado/rechazado). Coincide funcionalmente con el Jefe de nutrición; cualquier nutricionista autorizado para la dieta puede ejercerlo. |
| **Paciente** | Recibe los menús asignados; consulta sus valoraciones nutricionales; solicita revalorizaciones de forma virtual especificando al nutricionista ejecutante. |
| **Administrador del sistema** | Rol de mantenimiento externo al dominio: previene la existencia de alimentos duplicados e impide la asignación doble de un menú. |

### Entidades y nomencladores del dominio

| Tipo | Elementos |
|---|---|
| **Entidades (11)** | Ítem alimentario, con sus dos subclases, Alimento y Plato; Dieta; Menú; Nutricionista; Paciente; Sala; Valoración (nutricional); Revalorización; Servicio (entidad débil de la asignación) |
| **Nomencladores (7)** | Grupo nutricional, Tipo de preparación *(entrante, plato fuerte, postre, bebida)*, Nivel calórico *(bajo, medio, alto)*, Restricción alimentaria, Programa de atención, Especialidad, Resultado nutricional |

El enunciado llama «alimento» tanto al alimento simple como al plato. En este informe, **ítem**

**alimentario** (o simplemente *ítem*) designa lo genérico, y «alimento» queda solo para el simple. Cada ítem —un **alimento** o un **plato**— se clasifica por grupo nutricional, tipo de preparación y nivel calórico, y puede contar con una **ficha nutricional ampliada** (macronutrientes, micronutrientes, alérgenos, modo de preparación). El número y el tipo de campos de esa ficha varían de un elemento a otro y no se ajustan a una estructura fija, de modo que cada uno almacena únicamente los atributos que le corresponden: por eso la ficha se gestiona de forma independiente del resto de sus datos. Cómo se implementa esa independencia es una decisión de diseño, no un requerimiento: la ficha reside en MongoDB y guarda la clave del ítem al que pertenece, sin que el modelo relacional la referencie. La justificación está en el ADR correspondiente del repositorio, y sus restricciones, en la §4.3.

Una **dieta** define los grupos nutricionales a cubrir durante el tratamiento (`CubreDieta`), las restricciones alimentarias a respetar (`Restringe`) y el programa de atención institucional al que corresponde. Los **menús** generados para una dieta almacenan los parámetros de generación como datos inmutables: cantidad total de ítems (`CantidadTotalAlimentos`) y origen automático (`EsAutomático`) como atributos escalares, y la proporción por tipo de preparación y cobertura de grupos nutricionales como relaciones propias (`Distribuye`, `CubreMenú`).

Cada día en que un menú asignado se sirve a un paciente queda registrado como un **servicio**. El **consumo** registra, en cada servicio, si el paciente aceptó o rechazó cada ítem del menú, sea un alimento o un plato completo. Las **valoraciones nutricionales** documentan, con su fecha, el resultado del estado del paciente ante un menú recibido, referenciando un valor del nomenclador `ResultadoNutricional`. Las **revalorizaciones** registran la solicitud virtual del paciente de revisión de una valoración previa, identificando el nutricionista ejecutante y, cuando ya fue atendida, la nueva valoración generada.

**Decisiones de modelado relevantes**

- **Generalización `ItemAlimentario`.** Alimento y plato comparten las cuatro clasificaciones (grupo nutricional, tipo de preparación, nivel calórico y nutricionista que lo describe) y su inclusión en el menú. Por eso son subclases de una superclase, `ItemAlimentario`, con especialización **total** (todo ítem es alimento o plato) y **disjunta** (es solo uno de los dos), distinguidas por el discriminador `Clase`. Las relaciones comunes se modelan una sola vez, contra el ítem. La composición, en cambio, queda entre las subclases: `Compone` relaciona un plato con los alimentos simples que lo forman, con su `Cantidad`. Un menú incluye ítems (`Incluye`).
- **Jefe de nutrición** es un rol de Nutricionista, no una entidad. Un menú recibe **a lo sumo un dictamen**. Su estado no se almacena, sino que se deriva de ese dictamen:
  - **pendiente**, si no tiene validación;
  - **final**, si su validación está aprobada;
  - **rechazado**, si no lo está.

  Como cada menú tiene un único dictamen, no hace falta comparar fechas entre revisores para saber cuál vale. Si el jefe rechaza un menú e indica confeccionar otro, el menú nuevo referencia la validación que lo originó (`Origina`).
- **Asignación** se modela como N:M (`Asignación(MenúId, PacienteId)`); la clave primaria compuesta garantiza la unicidad por par.
- **Compatibilidad alimentaria** se resuelve con dos vías independientes. `CubreDieta` relaciona la dieta con los grupos nutricionales que le conciernen y califica cada uno con el atributo `Tipo`: *Requerido* si la dieta debe cubrirlo a lo largo del tratamiento, *Restringido* si lo prohíbe por un criterio propio de esa dieta. `Restringe + Excluye` aporta la segunda vía: las restricciones alimentarias son un catálogo institucional, y cada una excluye los grupos que le correspondan, una sola vez, para todas las dietas que la declaren. Los **grupos prohibidos** de una dieta son la unión de las dos vías; sus **grupos requeridos**, los de `Tipo` *Requerido*. Que un grupo quede prohibido por ambas vías es válido: la prohibición se repite sin contradecirse. Lo que no se admite es que un grupo sea a la vez requerido y excluido por una restricción de la misma dieta (RS-11).
- **Parámetros de generación**: `CantidadTotalAlimentos` y `EsAutomático` son escalares de Menú; proporción y cobertura son relaciones (`Distribuye`, `CubreMenú`). `Cubre` se desambigua en `CubreDieta` y `CubreMenú`. `CantidadTotalAlimentos` y las proporciones de `Distribuye` se interpretan sobre los ítems incluidos: un plato cuenta como un ítem, sin descomponerse en sus alimentos.
- **`ResultadoNutricional`** es un nomenclador con `Nombre` y `Valor` únicos, evitando texto libre y habilitando cálculo de promedios.
- **Servicio y consumo.** El consumo se registra por servicio:
  - **`Servicio`** es una entidad débil de la asignación, identificada por el día en que el menú se sirvió (`FechaServicio`).
  - **`Consume`** relaciona el servicio con cada ítem del menú y registra su `Aceptación`. Como el consumo es por ítem, el rechazo de un plato completo se registra como tal.
  - **Supuesto:** un menú asignado se sirve a lo sumo una vez por día al mismo paciente.

- **`FechaValoración`.** La valoración tiene fecha propia, de modo que las valoraciones de un paciente se ordenan sin depender de la fecha de creación del menú.
- **El solicitante de una revalorización se deriva.** Es el paciente de la valoración revisada; registrarlo aparte repetía un dato, así que no se almacena ni se dibuja (RS-09).

**Catálogo de alimentos y platos**

| # | Requerimiento |
|---|---|
| RF-01 | El sistema permite registrar alimentos con nombre único, grupo nutricional, tipo de preparación, nivel calórico y nutricionista descriptor. |
| RF-02 | El sistema impide el registro de ítems con nombre duplicado (normalizado), sean alimentos o platos. |
| RF-03 | El sistema permite registrar platos como composiciones de alimentos con cantidades, clasificados con los mismos atributos que los alimentos. |
| RF-04 | El sistema permite asociar a cada alimento y a cada plato una ficha nutricional ampliada (macronutrientes, micronutrientes, alérgenos, modo de preparación), cuyos campos varían de un elemento a otro, y consultarla o modificarla de forma independiente del resto de sus datos. |

**Usuarios y roles**

| # | Requerimiento |
|---|---|
| RF-05 | El sistema gestiona nutricionistas con especialidad, y pacientes con sala de hospitalización asignada. |
| RF-06 | El sistema controla qué nutricionistas están autorizados para crear y validar menús de cada dieta. |
| RF-07 | El sistema permite a cada nutricionista consultar los menús de las dietas en las que está autorizado, cualquiera sea el nutricionista que los creó. |
| RF-08 | El sistema registra la inscripción de pacientes en dietas. |

> **RF-07 no requiere estructura nueva.** «Atender la misma dieta» es estar autorizado en ella, de modo que los menús visibles para un nutricionista se obtienen recorriendo `Nutricionista` → `Autoriza` → `Dieta` ← `PerteneceDieta` ← `Menú`. La autorización, que RF-06 ya exige para crear y validar, delimita también lo que se puede consultar.

**Dietas y menús**

| # | Requerimiento |
|---|---|
| RF-09 | El sistema permite definir dietas con programa de atención, grupos nutricionales a cubrir y restricciones alimentarias a respetar. |
| RF-10 | El sistema soporta la generación automática de menús aplicando parámetros inmutables: proporción por tipo de preparación, cobertura de grupos nutricionales y cantidad total de ítems. |
| RF-11 | El sistema permite la confección manual de menús bajo criterios del jefe de nutrición. Cuando el jefe rechaza un menú e indica confeccionar otro, el menú nuevo queda vinculado al dictamen rechazado que lo originó. |
| RF-12 | El sistema registra la validación de un menú con fecha, dictamen (aprobado/rechazado) y observaciones, garantizando que el validador no sea el creador. |

| # | Requerimiento |
|---|---|
| RF-13 | El sistema asigna menús aprobados (finales) a pacientes inscritos en la dieta correspondiente, impidiendo la asignación doble. |

**Registro clínico y valoraciones**

| # | Requerimiento |
|---|---|
| RF-14 | El sistema registra cada día en que un menú asignado se sirve al paciente y, en cada servicio, si el paciente aceptó o rechazó cada ítem del menú, alimento o plato. |
| RF-15 | El sistema registra valoraciones nutricionales del estado del paciente, con su fecha, referenciando un valor del nomenclador de resultados y el nutricionista emisor. |
| RF-16 | El sistema procesa solicitudes virtuales de revalorización, identificando el nutricionista ejecutante y la valoración revisada, y vincula la nueva valoración cuando se completa. |

**Requerimientos no funcionales**

| # | Requerimiento |
|---|---|
| RNF-01 | Las consultas de disponibilidad calórica de una dieta y de valoraciones más recientes de un paciente responden de forma prácticamente inmediata, incluso en los horarios de mayor concurrencia. |

RNF-01 recoge una exigencia explícita del enunciado: esas dos consultas las repiten nutricionistas y pacientes a lo largo de toda la jornada, de modo que su latencia no puede depender de la carga del motor relacional. Se tratan en §2.2, bajo *Consultas recurrentes*, junto con la estrategia que las sostiene.

## 2.2 Requerimientos informacionales

El sistema debe dar respuesta a seis consultas informacionales y a dos consultas recurrentes. Las primeras son los reportes que el enunciado enumera como funcionalidades; las segundas son las peticiones que se repiten durante la jornada y están sujetas a RNF-01. Las seis reformulaciones aplican dos criterios. El primero: `CantidadTotalAlimentos` y `Distribuye` cuentan los ítems incluidos del menú, y un plato cuenta una vez. El segundo: se conserva el significado de cada reporte de la versión anterior del informe, expresado con el modelo actual, salvo donde el modelo nuevo obliga a cambiarlo, y en ese caso se explica por qué. A continuación se presenta el enunciado original de cada una y su reformulación enriquecida.

---

**Consulta 1 — Menús generados automáticamente por dieta**

> *«Obtener el listado de menús generados automáticamente para una dieta específica, indicando el nombre del creador, la fecha de creación y los parámetros utilizados.»*

**Reformulación:** Dado el identificador de una dieta, el sistema recupera el historial auditable de todos sus menús con `EsAutomático = TRUE`. Por cada uno proyecta: el nombre del nutricionista creador (quien ejecutó la parametrización algorítmica, no necesariamente quien ingresó los ítems al banco), la marca temporal de creación (`FechaCreación`) y el detalle completo de los parámetros almacenados de forma inmutable junto al menú: cantidad total de ítems (`CantidadTotalAlimentos`), proporciones por tipo de preparación (`Distribuye`) y grupos nutricionales cubiertos (`CubreMenú`). La cantidad y las proporciones se refieren a los ítems incluidos en el menú: un plato cuenta como uno, sin descomponerse en sus alimentos. Los resultados son ordenables por cualquier columna y exportables a PDF.

---

**Consulta 2 — Alimentos más utilizados en los menús finales de una dieta**

> *«Obtener los alimentos más utilizados en los menús finales de una dieta, clasificados por nivel calórico y grupo nutricional.»*

**Reformulación:** Dado el identificador de una dieta, el sistema filtra los **menús finales** — aquellos cuya única validación está aprobada — y sobre ellos cuenta cuántos menús distintos contienen cada alimento simple (presencia, no cantidad servida). Para esta consulta, un plato incluido se descompone por `Compone`: un alimento puede llegar directamente por `Incluye` o como componente de un `Plato` incluido por `Incluye`. Cada alimento cuenta una sola vez por menú aunque llegue por ambas vías. El resultado es una tabla de alimentos simples con número de apariciones, nivel calórico y grupo nutricional, ordenable y filtrable por top-N, exportable a PDF.

---

**Consulta 3 — Menús validados por un revisor determinado**

> *«Listar los menús que fueron validados por un revisor determinado, indicando la fecha de validación y las observaciones hechas durante el proceso.»*

**Reformulación:** Dado el identificador de un nutricionista en rol de revisor, el sistema recupera todos los dictámenes que registró: por cada uno reporta el menú evaluado (con su dieta y nutricionista creador), la fecha/hora del dictamen, las observaciones y el resultado (aprobado o rechazado). Cada menú tiene a lo sumo un dictamen; la consulta reúne, por tanto, los dictámenes de todos los menús revisados por ese nutricionista. También lista los rechazos, porque sus observaciones forman parte de la auditoría clínica; cuando un rechazo originó un reemplazo, puede mostrar el menú nuevo mediante `Origina`. Los resultados son ordenables y exportables a PDF.

---

**Consulta 4 — Desempeño nutricional de los pacientes ante un menú**

> *«Generar un reporte sobre el desempeño nutricional de los pacientes ante un menú, clasificando los alimentos por nivel calórico y comparando las tasas de aceptación.»*

**Reformulación:** Dado un menú, el sistema recupera sus servicios y los consumos de cada ítem incluido —alimento simple o plato— registrados en ellos. Por cada ítem calcula: número de servicios con consumo registrado, número de aceptaciones, número de rechazos y **tasa de aceptación** (aceptaciones ÷ servicios con consumo registrado). La tasa se pondera por servicio, no por paciente: cada registro de `Consume` corresponde a un ítem en un servicio determinado. A diferencia de la versión anterior, que descomponía los platos en sus alimentos, la tasa se calcula por ítem: el consumo se registra por ítem, de modo que el rechazo de un plato completo no se puede atribuir a sus ingredientes. Los resultados se clasifican por el nivel calórico del ítem y se presentan en tabla y gráfico comparativo. Las asignaciones o servicios sin consumo registrado quedan fuera del denominador; las tasas son derivadas y `Aceptación` en `Consume` es el único dato almacenado.

---

**Consulta 5 — Comparación de menús entre dietas**

> *«Comparar los menús generados para diferentes dietas, verificando la distribución de alimentos por grupo nutricional y nivel calórico y si los criterios de equilibrio fueron cumplidos.»*

**Reformulación:** Dadas dos o más dietas, el sistema contrasta sus menús en tres dimensiones: (1) distribución porcentual de ítems por grupo nutricional; (2) distribución por nivel calórico; (3) verificación de criterios de equilibrio en dos niveles: a nivel de dieta, si los grupos que exige cubrir (`CubreDieta` con `Tipo` *Requerido*) están presentes en el menú y si los que tiene prohibidos están ausentes; a nivel de parámetros de generación, si las proporciones por tipo de preparación y la cantidad total de ítems respetan lo declarado en `Distribuye` y `CantidadTotalAlimentos`. En las tres dimensiones, un plato incluido cuenta como un ítem con su propia clasificación, sin descomponerse en sus alimentos. Los grupos prohibidos de una dieta son la unión de dos vías: los excluidos por sus restricciones (`Restringe + Excluye`) y los que su propia cobertura marca como *Restringido*.

Los cuatro criterios pueden incumplirse: ninguno es una restricción que la base imponga (§4.5), y de ahí que el enunciado pida verificar «si fueron cumplidos». Ningún resultado se almacena: proporciones, tasas y dictámenes de cumplimiento son todos derivados.

---

**Consulta 6 — Correlación nivel calórico – resultado nutricional**

> *«Determinar, para cada dieta, la correlación entre el nivel calórico de los alimentos y el resultado nutricional promedio de los pacientes. La consulta debe identificar los 10 alimentos con la tasa de rechazo más alta, el nutricionista que los creó y la dieta a la que pertenecen. Además, debe comparar el desempeño nutricional de los pacientes que solicitaron una revalorización con el promedio general de valoraciones de sus respectivas salas para esas mismas dietas.»*

**Reformulación:** La consulta tiene tres partes independientes:

- **Parte A — Correlación por dieta:** para cada dieta, une dos ramas del modelo: la rama alimentaria (dieta → menús → ítems con nivel calórico) y la rama clínica (pacientes con valoraciones de las asignaciones de esos menús). La unión se hace por el par `(MenúId, PacienteId)` de la asignación; el resultado es el promedio del valor numérico de `ResultadoNutricional` segmentado por nivel calórico. Si hace falta acotar un período, la valoración y el servicio tienen fecha propia (`FechaValoración`, `FechaServicio`).
- **Parte B — Top-10 ítems rechazados:** sobre todos los registros de `Consume` del sistema, calcula la tasa de rechazo por ítem y selecciona los 10 con mayor tasa. Incluye alimentos simples y platos, porque el consumo registra ambos como ítems y puede reflejar el rechazo de un plato completo. Por cada uno reporta el nutricionista que lo **describió** en el banco (no el creador del menú) y la dieta a la que pertenece de forma indirecta, vía los menús en que aparece el consumo rechazado.
- **Parte C — Revalorizados frente al promedio de su sala:** para cada paciente que solicitó una revalorización ya atendida, compara el valor numérico de la valoración que esa revalorización generó con el promedio de las valoraciones de los demás pacientes de la misma sala sobre menús de la misma dieta. El paciente es el de la asignación que evalúa la valoración revisada, y la dieta, la del menú de esa asignación: `Revalorización` → `Valoración revisada` → `Asignación` → `Paciente` y `Revalorización` → `Valoración revisada` → `Asignación` → `Menú` → `Dieta`. Las revalorizaciones que todavía no generaron su valoración no entran en la comparación.

---

**Consultas recurrentes**

> *«Dado que la disponibilidad calórica de las dietas y las valoraciones más recientes son consultadas de forma reiterada por nutricionistas y pacientes a lo largo de toda la jornada, el sistema debe garantizar respuestas prácticamente inmediatas ante esas peticiones recurrentes, incluso en los horarios de mayor concurrencia.»*

A diferencia de las seis anteriores, estas dos no son reportes que se piden de vez en cuando, sino consultas de fondo que acompañan el trabajo diario. El enunciado no las define, así que el primer paso es precisar qué significan en términos del modelo.

**R1 — Disponibilidad calórica de una dieta.** Para cada nivel calórico, cuántos ítems del banco son compatibles con la dieta y cómo se distribuyen por nivel calórico los ítems de sus menús finales. Un ítem es compatible cuando su grupo nutricional no está prohibido para esa dieta, es decir, cuando no lo excluye ninguna de sus restricciones (`Restringe + Excluye`) ni lo marca como *Restringido* su propia cobertura (`CubreDieta`). Es el dato que un nutricionista consulta antes de confeccionar un menú: con qué cuenta, y en qué franja calórica.

> La disponibilidad se expresa en los tres niveles del nomenclador `NivelCalórico` —bajo, medio y alto—, no en kilocalorías. El modelo relacional no almacena ningún valor energético: los macronutrientes pertenecen a la ficha nutricional (§4.3), cuya estructura es variable y no se presta a agregación. La alternativa de que cada dieta declarase los niveles calóricos que admite, mediante una relación `Dieta`–`NivelCalórico`, se descartó: el enunciado habla de lo que está *disponible* para la dieta, no de un límite que la dieta imponga.

**R2 — Valoraciones más recientes de un paciente.** Las N valoraciones del paciente con mayor `FechaValoración`, con su resultado nutricional y el menú evaluado. Ordenarlas es posible porque la valoración tiene fecha propia; el modelo no depende para ello de la fecha de creación del menú.

**Trayectoria.** R1 recorre `Dieta` → `Restringe` → `Excluye` → `GrupoNutricional` y `Dieta` → `CubreDieta` para obtener los grupos prohibidos, `ItemAlimentario` → `PerteneceGrupo` → `GrupoNutricional` para descartar los ítems de grupos prohibidos, `ItemAlimentario` → `NivelCalórico` para clasificar el banco, y `Dieta` ← `Menú` (final) → `Incluye` → `ItemAlimentario` → `NivelCalórico` para la parte de los menús. R2 recorre `Paciente` ← `Asignación` ← `Valoración` → `ResultadoNutricional`, ordenando por `FechaValoración`. Ninguna de las dos almacena su resultado: ambas son derivadas.

**Cómo se garantiza la respuesta inmediata (RNF-01).** Las dos consultas son costosas en relación con su frecuencia —R1 recorre el banco completo y R2 ordena el historial del paciente— y sus resultados cambian poco entre una petición y la siguiente. Por eso se resuelven con un **caché de tipo clave-valor en memoria, en Redis, por delante de PostgreSQL**, que sigue siendo la única fuente de verdad:

| | Disponibilidad calórica | Valoraciones recientes |
|---|---|---|
| Clave | `dieta:{DietaId}:disponibilidad-calorica` | `paciente:{PacienteId}:valoraciones-recientes` |
| Valor | Conteo de ítems por nivel calórico | Las N valoraciones más recientes, ya ordenadas |

| | Disponibilidad calórica | Valoraciones recientes |
|---|---|---|
| Se invalida al | Alta o baja de un ítem del banco, o cambio de su nivel calórico o de su grupo; cambio en la cobertura o en las restricciones de la dieta; cambio en los grupos que excluye una restricción, que alcanza a todas las dietas que la declaran; aprobación de un menú suyo | Registrar una valoración de ese paciente, sea la original o la que genera una revalorización |

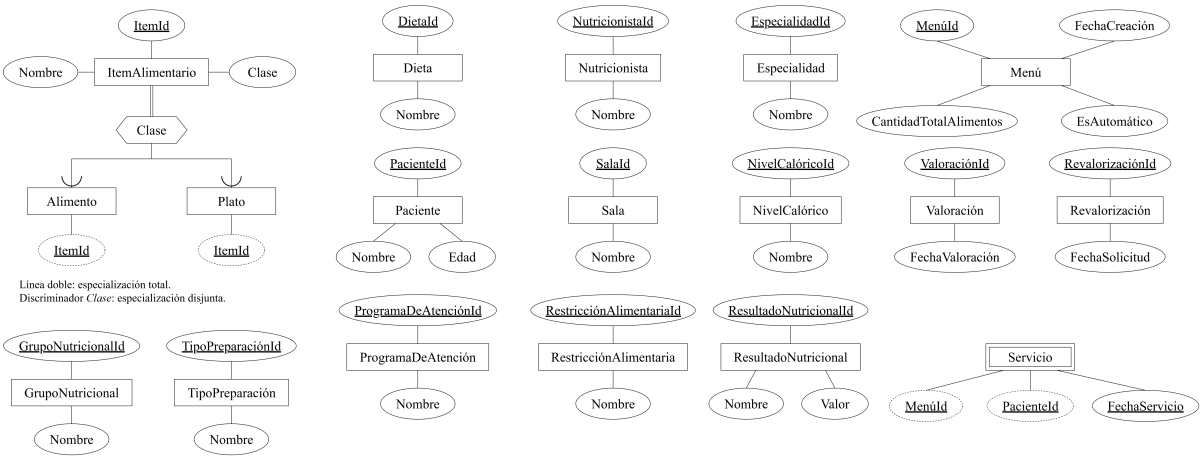
El patrón es *cache-aside*: la consulta busca primero en el caché y, si no encuentra la clave, la calcula contra PostgreSQL y la guarda. Las claves llevan además un tiempo de vida como respaldo, de modo que un fallo en la invalidación se corrige solo. Si Redis no está disponible, la consulta se resuelve contra PostgreSQL: se pierde la latencia, no la corrección. La elección de Redis y el reparto de responsabilidades entre los tres motores se justifican en el ADR correspondiente del repositorio.

# 3. MERX de la Base de Datos

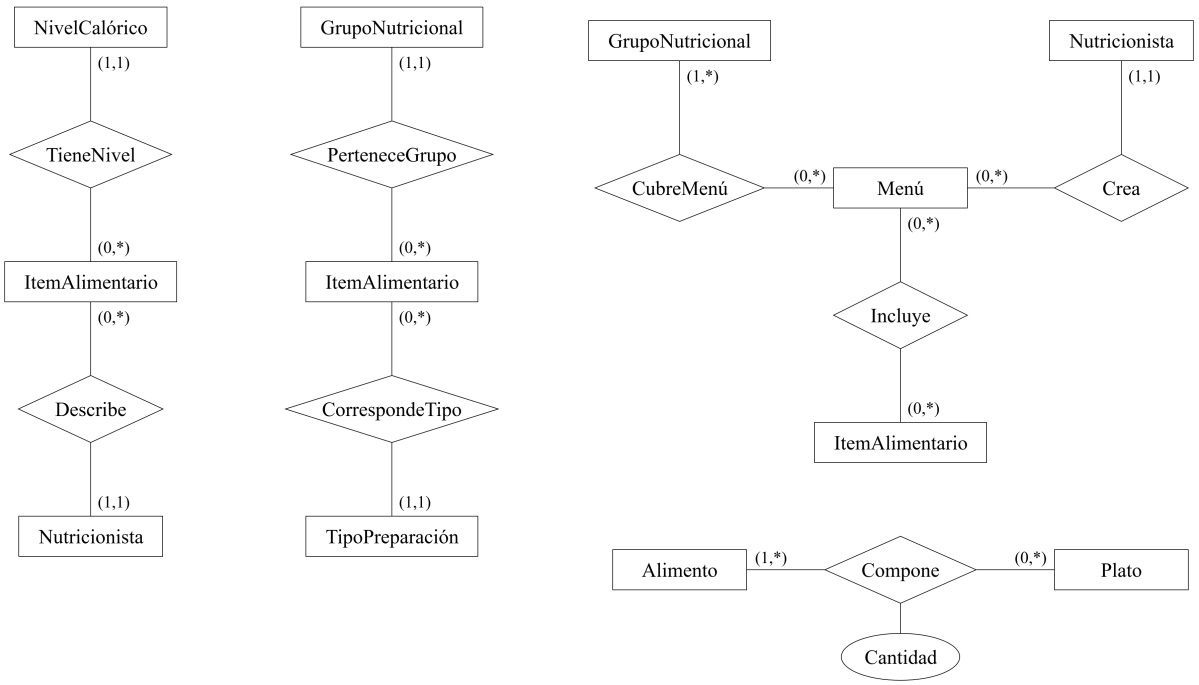
El Modelo Entidad-Relación Extendido (MERX) consolidado del sistema se presenta a continuación, en cuatro láminas: una con las entidades y sus atributos, y tres con las relaciones, agrupadas por área. Cinco convenciones ayudan a leerlo:

- **Cardinalidad.** El par `(mínimo, máximo)` junto a una entidad indica cuántas instancias de esa entidad se asocian a una sola instancia de la otra. Por ejemplo, `(0,1)` junto a `Nutricionista` en `Valida` significa que un menú recibe a lo sumo un dictamen.
- **Agregación.** Una relación se envuelve en una agregación solo cuando participa a su vez en otra relación, y sus atributos, si los tiene, cuelgan del rombo. Por eso quedan dos agregaciones:
  - `Validación`, que participa en `Origina`;
  - `Asignación`, que participa en `Evalúa` y en `Sirve`.
- **Especialización.** La de `ItemAlimentario` es total y disjunta. Que es total lo indica la línea doble entre la superclase y el discriminador; que es disjunta, el discriminador `Clase`. La notación del curso no fija cómo marcar la totalidad, así que la lámina de entidades lleva una leyenda.
- **Entidad débil.** `Servicio` (rectángulo de borde doble) depende de la agregación `Asignación` por la relación identificadora `Sirve` (rombo de borde doble). Su llave es la heredada de la asignación más `FechaServicio`.
- **Atributos heredados.** Los óvalos punteados son atributos heredados: los que una agregación toma de las entidades que envuelve, los que una entidad débil toma de su dueña y los que una subclase toma de su superclase. Los subrayados forman parte de la llave.

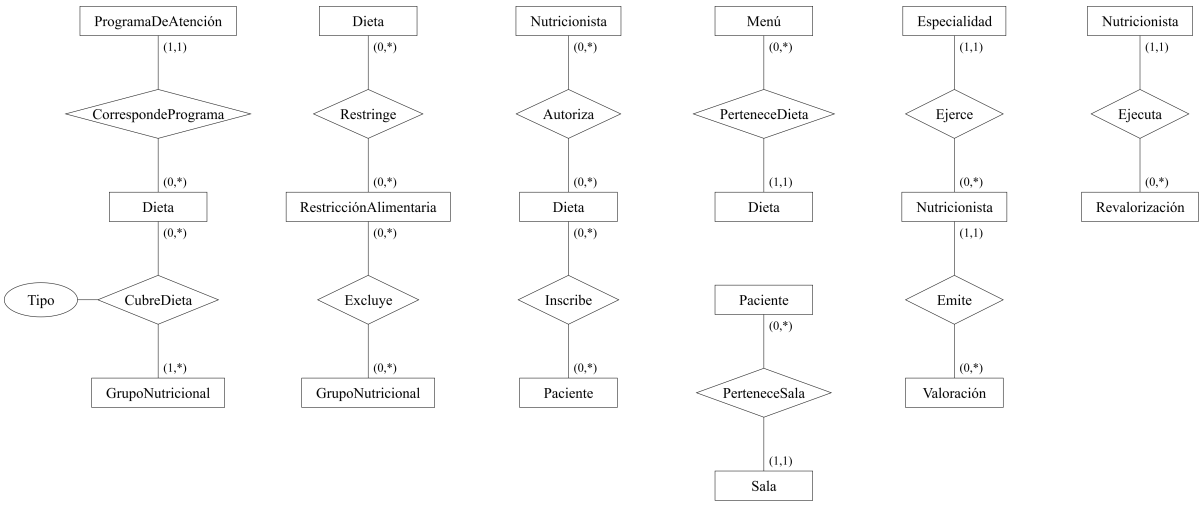
Cada relación tiene un nombre único: cuando dos comparten verbo, se distinguen por el participante (`PerteneceGrupo`, `PerteneceDieta`, `PerteneceSala`, `CubreDieta`, `CubreMenú`, …).



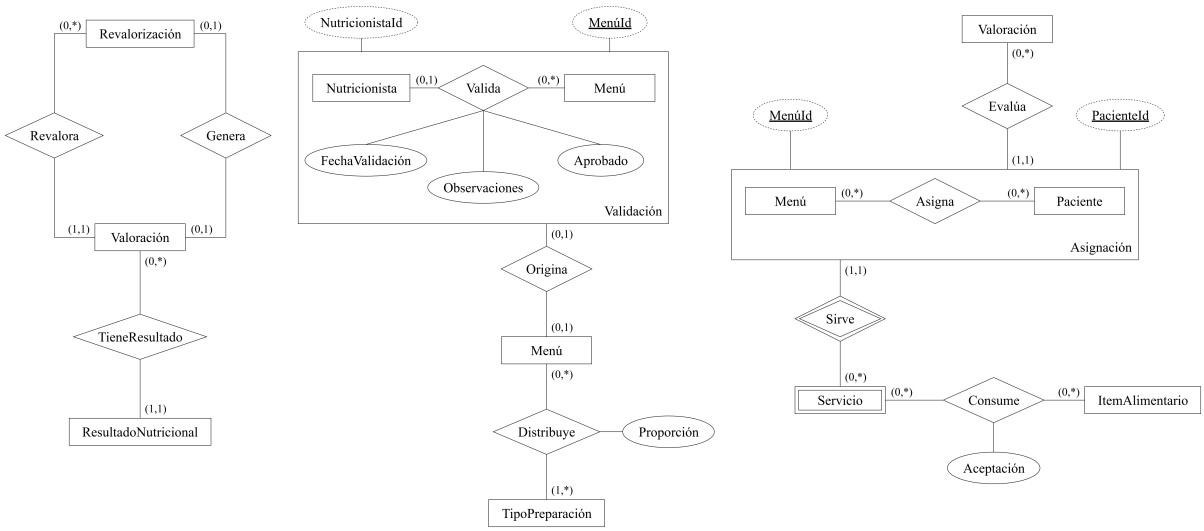

*MERX — Entidades y nomencladores*




*MERX — Relaciones 1: clasificación de los ítems y contenido del menú*




*MERX — Relaciones 2: dieta, autorizaciones y personas*




*MERX — Relaciones 3: validación, distribución, asignación, servicio y valoración*

# 4. Especificación de las restricciones de integridad establecidas y/o detectadas

Las restricciones se agrupan por su origen: **clave**, **referenciales**, **dominio**, **cardinalidad** y **semánticas**. Cada grupo se implementa con un mecanismo propio, y las excepciones se señalan aquí:

- **Clave (RC):** clave primaria o índice único. RC-01 usa un índice único sobre el nombre normalizado.
- **Referenciales (RR):** claves foráneas, con la acción de borrado que fija RX-02.
- **Dominio (RD):** tipo de la columna, `NOT NULL` y `CHECK`.
- **Cardinalidad (RK):** claves foráneas obligatorias, claves primarias e índices únicos. Las cardinalidades mínimas mayores que cero (RK-03, RK-07 y RK-09) se comprueban con triggers diferidos.
- **Semánticas (RS):** triggers de PostgreSQL, con estas excepciones:
  - RS-01 es el índice único de RC-01;
  - RS-09 se cumple por estructura;
  - RS-19 es un `CHECK`;
  - RS-02 y RS-08 quedan en la capa de aplicación: RS-08 la comprueba el dominio al crear el menú, y RS-02 corresponde a los casos de uso.
- **Ficha nutricional (RM):** en MongoDB, el validador `$jsonSchema` de la colección y un índice único (RM-08). En la capa de aplicación, las comprobaciones al guardar y al borrar una ficha, y una tarea programada de conciliación (RM-10).
- **Consolidación (RX):** RX-01 es una clave foránea compuesta, y RX-02 es la acción `ON DELETE` de cada clave foránea.

## 4.1 Restricciones estructurales

### 4.1.1 Clave

| # | Restricción |
|---|---|
| RC-01 | `ItemId` identifica unívocamente a cada ítem alimentario; alimentos y platos lo heredan de la superclase. El nombre normalizado es clave candidata: no hay dos ítems con el mismo nombre, sean alimentos o platos. |
| RC-02 | `GrupoNutricionalId`, `TipoPreparaciónId` y `NivelCalóricoId` identifican unívocamente a cada valor de su nomenclador. |
| RC-03 | Un mismo alimento aparece a lo sumo una vez en la composición de un plato: el par (`PlatoId`, `AlimentoId`) identifica cada ocurrencia de `Compone`. Ambos son nombres de rol de `ItemId`. |
| RC-04 | `DietaId` identifica unívocamente a cada dieta, `NutricionistaId` a cada nutricionista, `PacienteId` a cada paciente y `SalaId` a cada sala. |
| RC-05 | `ProgramaDeAtenciónId`, `RestricciónAlimentariaId` y `EspecialidadId` identifican unívocamente a cada valor de su nomenclador. |
| RC-06 | El par (`NutricionistaId`, `DietaId`) identifica cada autorización. |
| RC-07 | El par (`PacienteId`, `DietaId`) identifica cada inscripción. |
| RC-08 | El par (`DietaId`, `GrupoNutricionalId`) y el par (`DietaId`, `RestricciónAlimentariaId`) identifican cada ocurrencia de `CubreDieta` y de `Restringe`, respectivamente. |

| # | Restricción |
|---|---|
| RC-09 | `MenúId` identifica unívocamente a cada menú, `ValoraciónId` a cada valoración y `RevalorizaciónId` a cada revalorización. |
| RC-10 | `ResultadoNutricionalId` identifica unívocamente a cada categoría del nomenclador `ResultadoNutricional`. |
| RC-11 | Un menú registra a lo sumo una proporción por tipo de preparación: el par (`MenúId`, `TipoPreparaciónId`) identifica cada ocurrencia de `Distribuye`. |
| RC-12 | Un menú recibe a lo sumo un dictamen: `MenúId` identifica cada validación. |
| RC-13 | Un menú se asigna a lo sumo una vez a un mismo paciente: el par (`MenúId`, `PacienteId`) identifica cada ocurrencia de `Asignación`. |
| RC-14 | Cada ítem tiene a lo sumo un registro de consumo por servicio: (`MenúId`, `PacienteId`, `FechaServicio`, `ItemId`) identifica cada ocurrencia de `Consume`. |
| RC-15 | Un ítem o un grupo nutricional aparece a lo sumo una vez en un mismo menú: los pares de `Incluye` (`MenúId`, `ItemId`) y de `CubreMenú` (`MenúId`, `GrupoNutricionalId`) son únicos. |
| RC-16 | El par (`RestricciónAlimentariaId`, `GrupoNutricionalId`) identifica cada ocurrencia de `Excluye`. |
| RC-17 | Un menú asignado se sirve a lo sumo una vez por día al mismo paciente: la terna (`MenúId`, `PacienteId`, `FechaServicio`) identifica cada servicio. `FechaServicio` es la llave parcial de la entidad débil. |

`Autoriza`, `Inscribe` y `Restringe` son tablas de interconexión sin atributos propios: su identidad es el par de claves primarias de los participantes. `CubreDieta` también se identifica por ese par, pero **sí tiene un atributo propio**, `Tipo`, que califica cada grupo de la dieta como *Requerido* o *Restringido* (§2.1, RD-16).

| # | Restricción |
|---|---|

### 4.1.2 Referenciales

| # | Restricción |
|---|---|
| RR-01 | Toda clasificación de un ítem referencia un valor existente del nomenclador correspondiente. |
| RR-02 | Toda ocurrencia de `Compone` referencia un plato y un alimento existentes. |
| RR-03 | Todo ítem referencia al nutricionista que lo describió. |
| RR-04 | No se elimina un valor de nomenclador mientras lo use algún ítem, ni un alimento mientras forme parte de algún plato. |
| RR-05 | Toda dieta corresponde a un programa de atención existente. |
| RR-06 | Toda ocurrencia de `CubreDieta` referencia una dieta y un grupo nutricional existentes. |
| RR-07 | Toda ocurrencia de `Restringe` referencia una dieta y una restricción alimentaria existentes. |
| RR-08 | Toda autorización referencia un nutricionista y una dieta existentes. |
| RR-09 | Toda inscripción referencia un paciente y una dieta existentes. |
| RR-10 | Todo paciente referencia la sala a la que pertenece. |
| RR-11 | Todo nutricionista referencia la especialidad que ejerce. |
| RR-12 | No se elimina un valor de nomenclador mientras lo use alguna dieta o algún nutricionista, ni una dieta mientras figure en alguna autorización o inscripción. |
| RR-13 | Todo menú referencia la dieta a la que pertenece y el nutricionista que lo creó. |
| RR-14 | Toda ocurrencia de `Incluye`, `CubreMenú` y `Distribuye` referencia un menú existente y el ítem o el valor de nomenclador correspondiente. |
| RR-15 | Toda ocurrencia de `Validación` referencia un nutricionista y un menú existentes. |
| RR-16 | Toda ocurrencia de `Asignación` referencia un menú y un paciente existentes, y todo servicio referencia una asignación existente. Toda ocurrencia de `Consume` referencia un servicio existente y un ítem **incluido en el menú servido**: la referencia es a la inclusión (`MenúId`, `ItemId`), no al ítem suelto (RX-01). |
| RR-17 | Toda valoración referencia la asignación que evalúa, el nutricionista que la emite y una categoría existente de `ResultadoNutricional`. |
| RR-18 | Toda revalorización referencia al nutricionista que la ejecuta y a la valoración que revisa; la valoración generada, cuando existe, también debe existir. El paciente solicitante no se referencia: es el de la valoración revisada (RS-09). |
| RR-19 | No se elimina un menú mientras tenga validación o asignaciones, ni una asignación mientras tenga servicios o valoraciones, ni un servicio mientras tenga consumos, ni una valoración mientras una revalorización la revise o la haya generado. |
| RR-20 | Toda ocurrencia de `Excluye` referencia una restricción alimentaria y un grupo nutricional existentes. |
| RR-21 | No se elimina una restricción alimentaria ni un grupo nutricional mientras figuren en `Excluye`. |
| RR-22 | No se elimina un nutricionista ni un paciente mientras tengan hechos asociados: ítems descritos, menús creados, validaciones, valoraciones emitidas, revalorizaciones ejecutadas, autorizaciones o inscripciones. Tampoco se elimina un ítem alimentario que figure en algún menú, ni un valor de nomenclador que use algún menú. |
| RR-23 | Todo menú de reemplazo referencia la validación que lo originó, que tiene que existir (`Origina`). Esa validación no se elimina mientras origine un menú. |

RR-04, RR-12, RR-19, RR-21, RR-22 y RR-23 son las reglas que impiden borrar un hecho del que dependen otros, y se implementan con `ON DELETE RESTRICT`. No son las únicas claves foráneas del esquema ni agotan la política de borrado: la política completa, clave foránea por clave foránea, está en RX-02 (§4.4).

### 4.1.3 Dominio

| # | Restricción |
|---|---|
| RD-01 | `TipoPreparación` toma exactamente los valores *entrante*, *plato fuerte*, *postre* y *bebida*. |
| RD-02 | `NivelCalórico` toma exactamente los valores *bajo*, *medio* y *alto*. |
| RD-03 | `GrupoNutricional` es un catálogo abierto; se exige únicamente que cada nombre sea único. |
| RD-04 | `Nombre` es obligatorio y no vacío en `ItemAlimentario` y los tres nomencladores. |
| RD-05 | `Cantidad` de `Compone` se expresa en gramos, como un decimal estrictamente positivo. |
| RD-06 | `Edad` es mayor o igual que cero. |
| RD-07 | `Nombre` es obligatorio y no vacío en `Dieta`, `Nutricionista`, `Paciente`, `Sala` y en los tres nomencladores. |
| RD-08 | `Nombre` es único dentro de cada nomenclador. |
| RD-09 | `Edad` se registra como un número entero de años. |
| RD-10 | `CantidadTotalAlimentos` es un entero estrictamente positivo. |
| RD-11 | `Proporción` de `Distribuye` está en el intervalo (0, 1]. |
| RD-12 | `EsAutomático` (menú) y `Aprobado` (validación) son booleanos obligatorios. |
| RD-13 | `Aceptación` de `Consume` toma exactamente los valores *aceptado* y *rechazado*. |
| RD-14 | `ResultadoNutricional` tiene exactamente cinco categorías — *Muy mala*, *Mala*, *Normal*, *Buena* y *Excelente* —, con `Valor` 1, 2, 3, 4 y 5 respectivamente; `Nombre` y `Valor` son únicos y obligatorios. |
| RD-15 | `FechaCreación`, `FechaValidación`, `FechaSolicitud`, `FechaValoración` y `FechaServicio` son obligatorias. `FechaServicio` es un día, sin hora. `Observaciones` es opcional. |
| RD-16 | `Tipo` de `CubreDieta` es obligatorio y toma exactamente los valores *Requerido* y *Restringido*. |
| RD-17 | `Clase` es obligatoria y toma exactamente los valores *Alimento* y *Plato*. |

Las restricciones de catálogo cerrado (RD-01, RD-02 y RD-14) se garantizan en dos niveles: la FK impide usar un valor ajeno al catálogo, y el CHECK sobre el propio catálogo impide ampliarlo.

### 4.1.4 Cardinalidad

| # | Restricción |
|---|---|
| RK-01 | Cada ítem tiene exactamente un grupo nutricional, un tipo de preparación y un nivel calórico. |
| RK-02 | Cada ítem tiene exactamente un nutricionista que lo describió. |
| RK-03 | Todo plato se compone de al menos un alimento: `(1,*)` del lado de `Alimento` en `Compone`. |
| RK-04 | Cada dieta corresponde a exactamente un programa de atención. |
| RK-05 | Cada paciente pertenece a exactamente una sala. |
| RK-06 | Cada nutricionista ejerce exactamente una especialidad. |
| RK-07 | Toda dieta exige cubrir al menos un grupo nutricional: tiene al menos una ocurrencia de `CubreDieta` con `Tipo` *Requerido*. De ahí el `(1,*)` del lado de `GrupoNutricional` en `CubreDieta`. |
| RK-08 | Cada menú tiene exactamente un creador y una dieta. |
| RK-09 | Todo menú tiene al menos una proporción por tipo de preparación y al menos un grupo nutricional cubierto. |
| RK-10 | Cada valoración evalúa exactamente una asignación, la emite exactamente un nutricionista y tiene exactamente un resultado. |
| RK-11 | Cada revalorización tiene exactamente un ejecutor y una valoración revisada, cuyo paciente es el solicitante. Genera a lo sumo una valoración, y cada valoración es generada por a lo sumo una revalorización. |

| # | Restricción |
|---|---|
| RK-12 | Cada ítem es exactamente un alimento o un plato: la especialización de `ItemAlimentario` es total y disjunta. |
| RK-13 | Cada menú recibe a lo sumo un dictamen: `(0,1)` del lado de `Nutricionista` en `Valida`. |
| RK-14 | Cada servicio pertenece a exactamente una asignación: `(1,1)` del lado de `Asignación` en `Sirve`. Una asignación puede tener cero o más servicios. |
| RK-15 | Una validación origina a lo sumo un menú de reemplazo, y un menú reemplaza a lo sumo una validación: `(0,1)` en los dos extremos de `Origina`. |

> Las restricciones de cardinalidad mínima ≥ 1 (RK-03, RK-07 y RK-09) no son expresables con claves foráneas: la entidad se registra antes que sus ocurrencias y la comprobación solo tiene sentido al cierre de la operación. En la capa de aplicación, el alta y su detalle se registran en una misma unidad de trabajo; en PostgreSQL se refuerzan con un trigger de restricción diferido (`DEFERRABLE INITIALLY DEFERRED`).

---

## 4.2 Restricciones semánticas

Las restricciones semánticas no acotan los valores que se pueden almacenar, sino **quién puede operar sobre ellos** y bajo qué condiciones.

| # | Restricción |
|---|---|
| RS-01 | No existen ítems duplicados: dos ítems, sean alimentos o platos, no tienen el mismo nombre normalizado (RC-01). |
| RS-02 | Un nutricionista solo clasifica los ítems que él mismo describió. |
| RS-03 | Un nutricionista solo genera o valida menús de las dietas en las que está autorizado. |
| RS-04 | Un paciente solo recibe menús de las dietas en las que está inscrito. |
| RS-05 | Un nutricionista no valida un menú que él mismo creó. |
| RS-06 | La fecha de validación no es anterior a la fecha de creación del menú validado. |
| RS-07 | La parametrización de un menú (`CantidadTotalAlimentos`, `EsAutomático`, `Distribuye`, `CubreMenú`) no cambia después de creado el menú. |
| RS-08 | La suma de las proporciones de un menú es 1. |
| RS-09 | El paciente que solicita una revalorización es el mismo de la valoración que revisa. Se cumple por estructura: el solicitante no se registra aparte, sino que se obtiene de la valoración revisada. |
| RS-10 | La valoración generada por una revalorización evalúa la misma asignación que la valoración revisada y la emite el nutricionista que ejecutó la revalorización. |
| RS-11 | Un grupo que una dieta exige cubrir (`CubreDieta` con `Tipo` *Requerido*) no puede estar excluido por una de sus restricciones (`Restringe` seguido de `Excluye`). |
| RS-12 | La composición tiene un solo nivel: en `Compone`, el compuesto es un plato y el componente es un alimento simple, nunca otro plato. |
| RS-13 | La clase de un ítem no cambia después de registrado. |
| RS-14 | Ningún servicio es anterior al día en que se aprobó el menú. |
| RS-15 | La validación de un menú no se modifica ni se borra: el primer dictamen cierra el menú. |
| RS-16 | Solo se asigna un menú final: un menú pendiente o rechazado no se asigna. |
| RS-17 | Solo un rechazo origina un menú de reemplazo: la validación de origen no está aprobada. Un menú final no se reemplaza. |
| RS-18 | El menú de reemplazo es de la misma dieta que el menú rechazado. |
| RS-19 | Un menú no es reemplazo de sí mismo. |

**RS-11 restringe la definición de la dieta, no el contenido de sus menús.** Una dieta que exige un grupo y a la vez lo prohíbe es incoherente: ningún menú podría cumplirla. Por eso solo cuenta `Tipo` *Requerido*; un grupo marcado *Restringido* que además esté excluido por una restricción repite la prohibición sin contradecirla. Que un **menú** incluya ítems de un grupo prohibido por su dieta es una cuestión distinta, y no es una restricción: se trata en §4.5.

---

## 4.3 Restricciones de la ficha nutricional (MongoDB)

Las restricciones siguientes pertenecen a la ficha nutricional, almacenada en MongoDB. Se listan por referencia, sin duplicar la especificación del documento correspondiente: se conserva el identificador y se resume en una línea. El mecanismo de cada una está en ese documento, y por grupo, al principio de esta sección.

| # | Restricción |
|---|---|
| RM-01 | Toda ficha identifica el ítem de PostgreSQL al que pertenece (`origen.id`) y su clase (`origen.tabla`). |
| RM-02 | `origen.tabla` solo puede ser `Alimento` o `Plato`: es la clase del ítem (RD-17). |
| RM-03 | `tipo` es `simple` o `compuesto`, coherente con la clase: `Alimento` → `simple`, `Plato` → `compuesto`; solo las fichas de plato llevan `ingredientes`. |
| RM-04 | Los valores numéricos de la porción son positivos y los alérgenos no se repiten. |
| RM-05 | Los bloques `macronutrientes`, `micronutrientes` y `modoPreparacion` no tienen campos obligatorios. |
| RM-06 | En la ficha de un plato, el conjunto de `ingredientes.alimentoId` coincide con los alimentos del plato en `Compone`. |
| RM-07 | Solo existe ficha para ítems existentes. |
| RM-08 | Cada fila tiene a lo sumo una ficha. |
| RM-09 | Al eliminar un alimento o un plato se elimina su ficha. |
| RM-10 | Periódicamente se eliminan las fichas huérfanas y se reconstruyen las copias de `nombre` desactualizadas. |

Con la generalización (§2.1), alimentos y platos están en una sola tabla, `ItemAlimentario`. El campo `origen.tabla` conserva el nombre que tiene en el diseño de la ficha, y guarda la clase del ítem.

---

## 4.4 Restricciones definidas en la consolidación

Dos restricciones quedaron sin especificar en los submodelos individuales y se definen en la consolidación.

| # | Restricción |
|---|---|
| RX-01 | Solo se registra consumo de los ítems incluidos en el menú servido (`Incluye`). Un plato se acepta o se rechaza completo, no por ingrediente. Se impone de forma declarativa: `Consume` referencia la inclusión (`MenúId`, `ItemId`), no el ítem suelto (RR-16), y por eso no se dibuja en el MERX. |
| RX-02 | Política de borrado entre submodelos: `CASCADE` únicamente en las seis claves foráneas con que las filas de composición o de parametrización referencian a su dueño; `RESTRICT` en todas las demás. El detalle está en las tablas siguientes. |

**RX-02, por qué así:** un hecho del que dependen otros no se puede borrar. Las filas que solo describen a su dueño, en cambio, desaparecen con él: así un menú que todavía no tiene dictamen se borra por completo, con su parametrización y sus ítems. Con esta política, la inmutabilidad de la parametrización (RS-07) bloquea los cambios sueltos y deja pasar solo el borrado en cascada.

El criterio distingue dos situaciones. Una fila **existe por su dueño y carece de sentido sin él** —la composición de un plato, la parametrización de un menú—: se borra con él, en `CASCADE`. Una fila **registra un hecho propio** —una validación, una asignación, un consumo— o **referencia un catálogo compartido**: el borrado se impide, en `RESTRICT`, porque perderla falsearía el historial o dejaría huérfanas a otras filas.

En las dos tablas que siguen, `A → B` es la clave foránea de A que referencia a B.

**Las seis claves foráneas en `CASCADE`:**

| Clave foránea | Por qué |
|---|---|
| `Compone → Plato` | Los ingredientes describen al plato; sin él no existen. |
| `Incluye → Menú` | Los ítems incluidos son el contenido del menú. |
| `Distribuye → Menú` | Parámetro de generación del menú. |
| `CubreMenú → Menú` | Parámetro de generación del menú. |
| `CubreDieta → Dieta` | La cobertura define a la dieta. |
| `Restringe → Dieta` | Las restricciones definen a la dieta. |

**Las claves foráneas en `RESTRICT`,** agrupadas por lo que protegen:

| Grupo | Claves foráneas | Por qué |
|---|---|---|
| Clasificación de los ítems | `ItemAlimentario` → `GrupoNutricional`, `TipoPreparación`, `NivelCalórico`, `Nutricionista` descriptor | No se borra un valor de nomenclador en uso (RR-04) ni el nutricionista que describió un ítem (RR-22). |
| Composición | `Compone` → `Alimento` | No se borra un alimento que es ingrediente de un plato (RR-04). |
| Uso de un ítem o un nomenclador | `Incluye` → `ItemAlimentario`; `Distribuye` → `TipoPreparación`; `CubreMenú` → `GrupoNutricional` | El borrado en cascada del menú no debe arrastrar el ítem ni el nomenclador que la fila referencia (RR-22). |
| Cobertura y restricciones de una dieta | `CubreDieta` → `GrupoNutricional`; `Restringe` → `RestricciónAlimentaria` | Lo mismo del lado de la dieta: su borrado en cascada no arrastra los nomencladores (RR-12). |

| Grupo | Claves foráneas | Por qué |
|---|---|---|
| Entidades base y sus nomencladores | `Dieta` → `ProgramaDeAtención`; `Nutricionista` → `Especialidad`; `Paciente` → `Sala` | No se borra un valor de nomenclador en uso (RR-12), ni una sala con pacientes (RR-10). |
| Autorizaciones e inscripciones | `Autoriza` → `Nutricionista`, `Dieta`; `Inscribe` → `Paciente`, `Dieta` | No se borra una dieta que figure en alguna autorización o inscripción (RR-12), ni el nutricionista o el paciente del vínculo (RR-22). |
| Exclusión por restricción | `Excluye` → `RestricciónAlimentaria`, `GrupoNutricional` | RR-21. |
| Origen del menú | `Menú` → `Dieta`, `Nutricionista` creador | El menú documenta quién lo creó y para qué dieta (RR-13, RR-22). |
| Ciclo de revisión y servicio | `Validación` → `Menú`; `Validación` → `Nutricionista` revisor; `Asignación` → `Menú`; `Asignación` → `Paciente`; `Servicio` → `Asignación`; `Consume` → `Servicio`; `Consume` → `Incluye` | Un menú dictaminado, asignado o servido no se borra: su historial clínico depende de él (RR-19), y tampoco el revisor ni el paciente (RR-22). La última impide además quitar del menú un ítem que ya tiene consumos. |
| Menú de reemplazo | `Menú` → `Validación` de origen | No se borra la validación que originó otro menú: se perdería la traza del rechazo (RR-23). |
| Valoración | `Valoración` → `Asignación`, `Nutricionista` emisor, `ResultadoNutricional` | La valoración documenta un hecho clínico sobre una asignación concreta, con su emisor y su resultado (RR-17, RR-19, RR-22). |
| Revalorización | `Revalorización` → `Nutricionista` ejecutor, `Valoración` revisada, `Valoración` generada | No se borra una valoración que una revalorización revisa o generó (RR-19). |

Ninguna fila de las dos tablas contradice a la otra: protegen claves foráneas distintas. En particular, que `Incluye → Menú` sea `CASCADE` no choca con que `Asignación → Menú` sea `RESTRICT`, porque un menú asignado no llega a borrarse.

---

| Condición que se admite |
|---|

## 4.5 Lo que deliberadamente no se restringe

| Condición que se admite |
|---|
| La clasificación de un plato no se deriva de sus ingredientes: un plato de nivel calórico bajo puede contener alimentos de nivel alto.<br>Un alimento puede no formar parte de ningún plato, y un valor de nomenclador puede no estar en uso.<br>Una dieta puede no declarar ninguna restricción alimentaria.<br>Un paciente puede no estar inscrito en ninguna dieta y un nutricionista puede no estar autorizado en ninguna.<br>Los nombres de `Dieta`, `Nutricionista`, `Paciente` y `Sala` no son únicos. Sí lo son el nombre normalizado de los ítems, porque alimentos y platos comparten un mismo espacio de nombres (RC-01, RS-01), y el de cada valor de nomenclador (RD-08).<br>`Especialidad` no se deriva de `Autoriza`: un nutricionista puede estar autorizado en una dieta ajena a su especialidad.<br>El traslado de un paciente entre salas actualiza la fila del paciente, sin historial.<br>La composición real de un menú no se contrasta con su parametrización.<br>El estado de un menú (pendiente, final o rechazado) no se almacena: se deriva de su único dictamen (§2.1).<br>La asignación no tiene fecha propia: lo que ocurre en el tiempo son sus servicios.<br>Una asignación puede no tener servicios ni valoraciones, y un servicio puede no tener consumos registrados.<br>El consumo no registra cantidades, solo la aceptación por ítem.<br>La correspondencia entre las restricciones alimentarias de una dieta y los alérgenos de sus ítems no se modela. La compatibilidad por grupo nutricional sí está en el esquema (`Restringe` y `Excluye`); la compatibilidad por alérgeno la comprueba la capa de aplicación contra la ficha nutricional, y no es una restricción de integridad: la base no impide el menú, lo advierte el sistema. Saber qué alérgeno excluye cada restricción es configuración de la aplicación (§5.6).<br>Un menú puede incluir ítems de grupos prohibidos por su dieta. Que no los incluya es un **criterio de evaluación**, no una restricción, y la consulta 5 lo verifica. La compatibilidad alimentaria se comprueba en dos niveles, y ninguno de los dos es una restricción de integridad: por **grupo nutricional**, que el modelo relacional resuelve con `Restringe` y `Excluye` sin consultar la ficha; y por **alérgeno**, que la capa de aplicación contrasta contra la ficha nutricional al generar o validar un menú, para advertir al nutricionista (fila anterior). Imponer el primero en la base, además, vaciaría la consulta 5: su propósito es verificar si los criterios de equilibrio fueron cumplidos, y una de sus comprobaciones es precisamente que ningún ítem del menú pertenezca a un grupo prohibido por su dieta. Si la base impidiera esos menús, la comprobación daría siempre el mismo resultado y la consulta no tendría nada que detectar. El control queda en tres filtros previos al paciente: la generación automática no incluye grupos prohibidos, el revisor lo comprueba antes de aprobar y la consulta 5 lo reporta. |

# 5. Valoración de la corrección del diseño de la base de datos

Un diseño relacional se considera teóricamente correcto cuando cumple tres condiciones: cada esquema de la descomposición alcanza **al menos** la Tercera Forma Normal (3FN), se preserva el conjunto completo de dependencias funcionales del dominio (PPDF) y la reconstrucción por join natural no introduce tuplas espurias (PLJ).

La Forma Normal de Boyce-Codd no forma parte del criterio: es una condición más fuerte que la 3FN y no siempre es alcanzable junto con la PPDF, de modo que exigirla descartaría diseños correctos. Aquí se verifica al final, esquema por esquema, como propiedad adicional.

## 5.1 Conjunto de dependencias funcionales extraído del MERX

El punto de partida no es el cubrimiento minimal, sino el conjunto **F** de dependencias funcionales que se lee directamente del modelo conceptual. El cubrimiento minimal se obtiene después, aplicando el algoritmo a F (§5.2).

### 5.1.1 Reglas de extracción

1. **Conjuntos de entidades.** Por cada entidad con clave K y atributos U, se añade `K → U`.
2. **Entidades débiles.** Su clave es la de su dueña (una entidad, o una agregación como en el caso de `Servicio`, que depende de `Asignación`) más su llave parcial. `Servicio` no tiene atributos fuera de esa clave, de modo que no aporta ninguna dependencia no trivial.
3. **Especialización.** El discriminador de una especialización es un atributo de la superclase, y en el diagrama se rotula además sobre la propia especialización. El de `ItemAlimentario` se llama `Clase`, distingue las dos subclases y aparece en F como el atributo `clase`, determinado por `id_item`. Las subclases no aportan dependencias propias, porque no tienen atributos que la superclase no tenga: lo que las distingue es la relación `Compone`, en la que un plato se compone de alimentos simples.
4. **Interrelaciones con atributos propios.** Por cada una con clave K, se añade `K → atributos`. La clave es compuesta salvo en `Valida`, cuya clave es el menú validado, porque un menú recibe a lo sumo un dictamen.
5. **Cardinalidad máxima 1.** Por cada extremo con cardinalidad máxima 1 se añade la dependencia del lado «muchos» hacia el lado «uno». Cuando la cardinalidad máxima es 1 en los dos extremos, como en `Genera` y `Origina` (RK-11, RK-15), la dependencia vale en los dos sentidos; el `UNIQUE` sobre la columna es su implementación (regla 6).
6. **Unicidades del dominio.** Las claves candidatas declaradas con `UNIQUE` aportan la dependencia inversa: el nombre de cada nomenclador y el de `ResultadoNutricional`, su `Valor`, y las dos relaciones 1:1 opcionales. En los ítems, lo único es el nombre **normalizado** (RC-01, RS-01). Eso implica que tampoco se repite el nombre tal como se escribe, y de ahí sale la dependencia `nombre_item → id_item`. La unicidad normalizada es más fuerte (compara sin distinguir mayúsculas ni espacios sobrantes), pero no es una dependencia entre atributos almacenados, así que queda como restricción.

### 5.1.2 Nombres de rol

La teoría de la normalización trabaja sobre un **esquema universal** R(U, F), y eso exige que un mismo nombre de atributo signifique lo mismo en todo U. En este modelo hay identificadores que aparecen con papeles distintos: `id_nutricionista` es a la vez quien describe un ítem, quien crea un menú, quien lo revisa, quien emite una valoración, quien ejecuta una revalorización y quien está autorizado en una dieta. Sin distinguirlos, el algoritmo deduciría dependencias falsas, por ejemplo que el nutricionista que describió un alimento es el que validó el menú que lo contiene.

Por eso, en U cada aparición con un papel distinto es un **atributo distinto**, y se denota con un sufijo de rol:

| Identificador | Papeles en U |
|---|---|
| `id_nutricionista` | `_descriptor` (ítem), `_creador` (menú), `_revisor` (validación), `_emisor` (valoración), `_ejecutor` (revalorización), `_autorizado` (`Autoriza`) |
| `id_grupo` | `_item` (clasificación), `_cubierto` (`CubreDieta`), `_cubierto_menu` (`CubreMenú`), `_excluido` (`Excluye`) |
| `id_dieta` | `_menu` (menú), `_cubierta`, `_restringida`, `_autorizada`, `_inscrita` |
| `id_menu` | `_origen` (`Origina`), `_validado`, `_distribuido`, `_incluyente`, `_cubierto`, `_asignado`, `_servido`, `_consumo`, `_valorado` |
| `id_paciente` | `_inscrito`, `_asignado`, `_servido`, `_consumo`, `_valorado` |
| `id_item` | `_incluido` (`Incluye`), `_consumido` (`Consume`); en `Compone`, `id_plato_compuesto` e `id_alimento_componente` |
| `id_tipo` | `_item` (clasificación), `_distribuido` (`Distribuye`) |
| `id_nivel` | `_item` (clasificación) |
| `id_restriccion` | `_dieta` (`Restringe`), `_excluyente` (`Excluye`) |
| `id_programa` | `_dieta` (la dieta corresponde a un programa) |
| `id_especialidad` | `_nutricionista` (el nutricionista ejerce una especialidad) |
| `id_sala` | `_paciente` (el paciente pertenece a una sala) |
| `id_resultado` | `_valoracion` (la valoración tiene un resultado) |
| `fecha_servicio` | la del servicio, y `fecha_servicio_consumo` la del consumo que se registra en él |
| `id_valoracion` | `_revisada`, `_generada` (revalorización) |

### 5.1.3 El conjunto F

**De las entidades:**

```
id_item            → clase, nombre_item, id_grupo_item, id_tipo_item,
                     id_nivel_item, id_nutricionista_descriptor
id_grupo           → nombre_grupo
id_tipo            → nombre_tipo
id_nivel           → nombre_nivel
id_programa        → nombre_programa
id_restriccion     → nombre_restriccion
id_especialidad    → nombre_especialidad
id_resultado       → nombre_resultado, valor_resultado
id_sala            → nombre_sala
id_dieta           → nombre_dieta, id_programa_dieta
```

```
id_nutricionista   → nombre_nutricionista, id_especialidad_nutricionista
id_paciente        → nombre_paciente, edad, id_sala_paciente
id_menu            → fecha_creacion, cantidad_total_alimentos, es_automatico,
                     id_dieta_menu, id_nutricionista_creador, id_menu_origen
id_valoracion      → id_menu_valorado, id_paciente_valorado,
                     id_nutricionista_emisor, id_resultado_valoracion,
                     fecha_valoracion
id_revalorizacion  → fecha_solicitud, id_nutricionista_ejecutor,
                     id_valoracion_revisada, id_valoracion_generada
```

**De las interrelaciones con atributos propios:**

```
(id_plato_compuesto, id_alimento_componente)                → cantidad
(id_menu_distribuido, id_tipo_distribuido)                  → proporcion
(id_dieta_cubierta, id_grupo_cubierto)                      → tipo
(id_menu_consumo, id_paciente_consumo, fecha_servicio_consumo,
 id_item_consumido)                                         → aceptacion
id_menu_validado → id_nutricionista_revisor, fecha_validacion, aprobado,
                   observaciones
```

La última merece atención: la clave de `Validación` **es el menú validado**, porque un menú recibe a lo sumo un dictamen (RC-12). Que un menú validado sea un menú, que el servicio en que se registra un consumo exista, que el ítem consumido esté incluido en el menú servido (RX-01) y que el origen de un menú de reemplazo sea una validación existente (RR-23) son **dependencias de inclusión** —claves foráneas, RR-15, RR-16 y RR-23—, no dependencias funcionales: es lo mismo que ocurre con todos los demás atributos que los nombres de rol separan.

**De las unicidades del dominio:**

```
nombre_grupo          → id_grupo
nombre_tipo           → id_tipo
nombre_nivel          → id_nivel
nombre_programa       → id_programa
nombre_restriccion    → id_restriccion
nombre_especialidad   → id_especialidad
nombre_resultado      → id_resultado
valor_resultado       → id_resultado
nombre_item           → id_item
id_valoracion_generada → id_revalorizacion
id_menu_origen         → id_menu
```

> Las dos últimas provienen de relaciones 1:1 **opcionales**: una revalorización puede no haber generado todavía su valoración, y un menú puede no ser reemplazo de ninguno. La dependencia se cumple sobre las filas donde el atributo está definido, que es lo que garantiza el `UNIQUE` sobre una columna que admite nulos. No afecta al resultado de la síntesis, porque en ambos casos la parte izquierda es equivalente a una clave que ya existe.

**Lo que no aporta dependencias no triviales.** Las siete interrelaciones M:N sin atributos propios (`Autoriza`, `Inscribe`, `Restringe`, `Excluye`, `Incluye`, `CubreMenú` y `Asignación`) y la entidad débil `Servicio` solo tienen la dependencia trivial K → K. Las interrelaciones 1:N sin atributos propios, como `Describe` o `PerteneceSala`, sí aportan dependencias: son las de la regla 5. En §5.3.2 se explica cómo entran sus atributos en la descomposición, porque **no pueden salir del algoritmo**: una dependencia trivial no forma parte de un cubrimiento minimal.

---

## 5.2 Cubrimiento minimal

Un cubrimiento minimal G de F cumple tres condiciones: toda parte derecha es un único atributo, ninguna parte izquierda tiene atributos extraños y ninguna dependencia es redundante. Se obtiene aplicando los tres pasos en ese orden.

### 5.2.1 Paso 1 — partes derechas unitarias

Cada dependencia con varios atributos a la derecha se descompone por la regla de descomposición. Por ejemplo:

```
id_paciente → nombre_paciente, edad, id_sala_paciente
```

se convierte en

```
id_paciente → nombre_paciente
id_paciente → edad
id_paciente → id_sala_paciente
```

Aplicado a todo F:

| Dependencia original | DF unitarias |
|---|---|
| `id_item → …` | 6 |

| Dependencia original | DF unitarias |
|---|---|
| `id_menu` → … (entidad) | 6 |
| `id_menu_validado` → … (validación) | 4 |
| `id_valoracion` → … | 5 |
| `id_revalorizacion` → … | 4 |
| `id_paciente` → … | 3 |
| `id_dieta` → …, `id_nutricionista` → …, `id_resultado` → … | 2 cada una |
| Las 7 restantes de entidad, las 4 de interrelación y las 11 de unicidad | 1 cada una |

El conjunto queda con **56 dependencias de parte derecha unitaria**.

### 5.2.2 Paso 2 — atributos extraños en la parte izquierda

Solo puede haberlos en las partes izquierdas compuestas, que son cuatro. Para cada una se calcula el cierre de cada subconjunto propio respecto de F y se comprueba que no contiene la parte derecha.

**(`id_plato_compuesto`, `id_alimento_componente`) → `cantidad`**

```
{id_plato_compuesto}⁺    = {id_plato_compuesto}          ∌ cantidad
{id_alimento_componente}⁺ = {id_alimento_componente}     ∌ cantidad
```

Ninguno de los dos basta: un plato tiene varios ingredientes con cantidades distintas, y un alimento aparece en varios platos con cantidades distintas. **No hay atributos extraños.**

**(`id_menu_distribuido`, `id_tipo_distribuido`) → `proporcion`** y **(`id_dieta_cubierta`, `id_grupo_cubierto`) → `tipo`** se resuelven igual: cada parte izquierda por separado deja la parte derecha fuera de su cierre, porque un menú tiene una proporción por cada tipo de preparación y una dieta califica varios grupos.

**(`id_menu_consumo`, `id_paciente_consumo`, `fecha_servicio_consumo`, `id_item_consumido`) → `aceptacion`**

Hay que descartar los cuatro subconjuntos de tres atributos. Ninguna parte izquierda de F está contenida en ninguno de ellos, de modo que su cierre es él mismo y no alcanza a `aceptacion`. Lo que sigue es la razón de dominio por la que F no contiene esas dependencias:

```
{id_menu_consumo, id_paciente_consumo, fecha_servicio_consumo}⁺  ∌ aceptacion
    el servicio registra la aceptación de cada ítem por separado
{id_menu_consumo, id_paciente_consumo, id_item_consumido}⁺       ∌ aceptacion
    el mismo ítem puede aceptarse un día y rechazarse otro
{id_menu_consumo, fecha_servicio_consumo, id_item_consumido}⁺    ∌ aceptacion
    dos pacientes pueden responder distinto ante el mismo ítem
{id_paciente_consumo, fecha_servicio_consumo, id_item_consumido}⁺ ∌ aceptacion
    el paciente puede recibir dos menús el mismo día
```

**No hay atributos extraños en ninguna de las cuatro.**

### 5.2.3 Paso 3 — dependencias redundantes

Una dependencia `X → A` es redundante si `A ∈ X⁺` calculado respecto de `G − {X → A}`. Las dependencias de entidad no lo son: cada atributo de sus partes derechas aparece como parte derecha en una sola dependencia de G (la excepción, `id_resultado`, se trata abajo con las de unicidad), de modo que, sin esa dependencia, ningún cierre lo alcanza.

Los casos que sí exigen comprobación son los de unicidad, porque conviven con la dependencia inversa:

`G − {nombre_grupo → id_grupo}:  {nombre_grupo}⁺ = {nombre_grupo}  ∌ id_grupo`

El nombre no determina nada más por otra vía, así que **no es redundante**. El mismo cálculo vale para los demás nomencladores, para `nombre_item` y para `valor_resultado`. En `ResultadoNutricional` conviven `nombre_resultado → id_resultado` y `valor_resultado → id_resultado`: ninguna deriva de la otra, porque de `nombre_resultado` no se alcanza `valor_resultado` sin pasar por `id_resultado`, y viceversa. Las dos se conservan, y la relación queda con tres claves candidatas.

El caso de la **revalorización** merece mención aparte, porque en el diseño anterior sí había una redundancia aquí. La tabla guardaba también el paciente solicitante, y RS-09 obligaba a que fuera el de la valoración revisada; eso equivale a `id_valoracion_revisada` → `id_paciente_solicitante`. Con ella, `id_revalorizacion` → `id_paciente_solicitante` es **transitiva**: redundante en el cubrimiento, y en la tabla implementada, una dependencia de un atributo no primo respecto de un determinante que no es clave, que la dejaba en 2FN. Al dejar de almacenarse el solicitante, esa dependencia desaparece de F y la restricción se cumple por construcción.

Ninguna otra dependencia resulta redundante. **G tiene las mismas 56 dependencias que salieron del paso 1.**

### 5.2.4 El cubrimiento minimal G

G tiene **56 dependencias, todas de parte derecha unitaria**. Se escriben una por una y numeradas, agrupadas por la entidad o la relación de la que salen:

```
g1   id_item                → clase
g2   id_item                → nombre_item
g3   id_item                → id_grupo_item
g4   id_item                → id_tipo_item
g5   id_item                → id_nivel_item
g6   id_item                → id_nutricionista_descriptor
g7   nombre_item            → id_item
g8   id_grupo               → nombre_grupo
g9   nombre_grupo           → id_grupo
g10  id_tipo                → nombre_tipo
g11  nombre_tipo            → id_tipo
g12  id_nivel               → nombre_nivel
g13  nombre_nivel           → id_nivel
g14  id_programa            → nombre_programa
g15  nombre_programa        → id_programa
g16  id_restriccion         → nombre_restriccion
g17  nombre_restriccion     → id_restriccion
g18  id_especialidad        → nombre_especialidad
g19  nombre_especialidad    → id_especialidad
g20  id_resultado           → nombre_resultado
g21  id_resultado           → valor_resultado
g22  nombre_resultado       → id_resultado
g23  valor_resultado        → id_resultado
g24  id_sala                → nombre_sala
g25  id_dieta               → nombre_dieta
g26  id_dieta               → id_programa_dieta
g27  id_nutricionista       → nombre_nutricionista
g28  id_nutricionista       → id_especialidad_nutricionista
g29  id_paciente            → nombre_paciente
g30  id_paciente            → edad
g31  id_paciente            → id_sala_paciente
g32  id_menu                → fecha_creacion
g33  id_menu                → cantidad_total_alimentos
g34  id_menu                → es_automatico
g35  id_menu                → id_dieta_menu
g36  id_menu                → id_nutricionista_creador
g37  id_menu                → id_menu_origen
g38  id_menu_origen         → id_menu
g39  id_menu_validado       → id_nutricionista_revisor
g40  id_menu_validado       → fecha_validacion
g41  id_menu_validado       → aprobado
g42  id_menu_validado       → observaciones
g43  id_valoracion          → id_menu_valorado
g44  id_valoracion          → id_paciente_valorado
g45  id_valoracion          → id_nutricionista_emisor
g46  id_valoracion          → id_resultado_valoracion
g47  id_valoracion          → fecha_valoracion
```

```
g48  id_revalorizacion      → fecha_solicitud
g49  id_revalorizacion      → id_nutricionista_ejecutor
g50  id_revalorizacion      → id_valoracion_revisada
g51  id_revalorizacion      → id_valoracion_generada
g52  id_valoracion_generada → id_revalorizacion
g53  (id_plato_compuesto, id_alimento_componente) → cantidad
g54  (id_menu_distribuido, id_tipo_distribuido) → proporcion
g55  (id_dieta_cubierta, id_grupo_cubierto) → tipo
g56  (id_menu_consumo, id_paciente_consumo, fecha_servicio_consumo, id_item_consumido)
       → aceptacion
```

---

## 5.3 Descomposición por el algoritmo de síntesis

El algoritmo de síntesis construye la descomposición a partir de **G**, no del cierre F*: agrupa las dependencias de G que comparten parte izquierda, o cuyas partes izquierdas son equivalentes, y crea un esquema por grupo. Formalmente, para cada grupo $X \to A_1, \ldots, X \to A_k$ se crea $R_i(U_i, F_i)$ con $U_i = X \cup \{A_1, \ldots, A_k\}$ y $F_i = \Pi_{U_i}(G^+)$, la proyección del cierre de G sobre $U_i$. Por último se eliminan los esquemas contenidos en otro. Agrupar por partes izquierdas equivalentes o por partes izquierdas idénticas da, en este caso, los mismos 20 esquemas: las partes izquierdas equivalentes (`id` y `nombre` en los nomencladores, por ejemplo) caen igual en un esquema que ya las contiene a las dos, y ese otro se elimina por estar contenido.

### 5.3.1 Esquemas obtenidos de G

Dos partes izquierdas son **equivalentes** cuando cada una determina a la otra, y en ese caso sus dependencias van al mismo esquema. Es lo que ocurre en los nomencladores, donde `id` y `nombre` se determinan mutuamente: no generan tablas separadas, sino una sola con dos claves candidatas.

| Esquema | Atributos | Claves candidatas |
|---|---|---|
| `ItemAlimentario` | id_item, clase, nombre_item, id_grupo_item, id_tipo_item, id_nivel_item, id_nutricionista_descriptor | id_item, nombre_item |
| `GrupoNutricional` | id_grupo, nombre_grupo | las dos |
| `TipoPreparación` | id_tipo, nombre_tipo | las dos |
| `NivelCalórico` | id_nivel, nombre_nivel | las dos |
| `ProgramaDeAtención` | id_programa, nombre_programa | las dos |
| `RestricciónAlimentaria` | id_restriccion, nombre_restriccion | las dos |
| `Especialidad` | id_especialidad, nombre_especialidad | las dos |
| `ResultadoNutricional` | id_resultado, nombre_resultado, valor_resultado | las tres |
| `Sala` | id_sala, nombre_sala | id_sala |
| `Dieta` | id_dieta, nombre_dieta, id_programa_dieta | id_dieta |
| `Nutricionista` | id_nutricionista, nombre_nutricionista, id_especialidad_nutricionista | id_nutricionista |

| Esquema | Atributos | Claves candidatas |
|---|---|---|
| `Paciente` | id_paciente, nombre_paciente, edad, id_sala_paciente | id_paciente |
| `Menú` | id_menu, fecha_creacion, cantidad_total_alimentos, es_automatico, id_dieta_menu, id_nutricionista_creador, id_menu_origen | id_menu; `id_menu_origen` es alternativa sobre las filas donde está definida |
| `Validación` | id_menu_validado, id_nutricionista_revisor, fecha_validacion, aprobado, observaciones | id_menu_validado |
| `Valoración` | id_valoracion, id_menu_valorado, id_paciente_valorado, id_nutricionista_emisor, id_resultado_valoracion, fecha_valoracion | id_valoracion |
| `Revalorización` | id_revalorizacion, fecha_solicitud, id_nutricionista_ejecutor, id_valoracion_revisada, id_valoracion_generada | id_revalorizacion; `id_valoracion_generada` es alternativa sobre las filas donde está definida |
| `Compone` | id_plato_compuesto, id_alimento_componente, cantidad | el par |
| `Distribuye` | id_menu_distribuido, id_tipo_distribuido, proporcion | el par |
| `CubreDieta` | id_dieta_cubierta, id_grupo_cubierto, tipo | el par |
| `Consume` | id_menu_consumo, id_paciente_consumo, fecha_servicio_consumo, id_item_consumido, aceptacion | la cuaterna |

Son **20 esquemas**: forman la descomposición ρ.

> **Por qué `Validación` sale como esquema propio.** Si `Valida` usara `id_menu` en lugar del rol `id_menu_validado`, su dependencia compartiría parte izquierda con la de la entidad `Menú` y el algoritmo las uniría en un solo esquema. Esa unión sería correcta, pero obligaría a dejar nulos `id_nutricionista_revisor`, `fecha_validacion`, `aprobado` y `observaciones` en todos los menús sin dictamen. Con el nombre de rol, que es la misma regla que se aplica al resto del modelo, los dos esquemas salen directamente del algoritmo y la ausencia de dictamen se representa por **ausencia de fila**. El estado del menú se deriva sin ambigüedad: pendiente si no hay fila en `Validación`, final si la hay con `aprobado`, rechazado si la hay sin él.

### 5.3.2 Las interrelaciones sin dependencias no triviales

Las ocho interrelaciones de §5.1.3 no aparecen en ningún esquema de los anteriores, y **no pueden aparecer**: sus únicas dependencias son triviales y un cubrimiento minimal no las contiene. Afirmar que «generan dependencias K → K» no las justifica; hay que explicar por dónde entran.

Entran por el esquema universal. **U contiene todos los atributos del MERX**, incluidos los pares de esas interrelaciones con sus nombres de rol. Ninguno de esos atributos aparece en la parte derecha de ninguna dependencia de G: nada determina qué dieta está autorizada a qué nutricionista, ni qué ítem incluye qué menú. Por el criterio del cálculo de claves, **un atributo que no figura en ninguna parte derecha pertenece a toda clave de U**, de modo que todos ellos están en la clave, y es el último paso del algoritmo el que los incorpora (§5.3.3).

### 5.3.3 El esquema de la llave y la descomposición final

El algoritmo de síntesis termina comprobando si algún $R_i$ contiene una clave candidata de U. Si ninguno la contiene, se añade un esquema formado por esa clave.

**Cálculo de la clave.** Se parte del **núcleo** N: los atributos que no aparecen en ninguna parte derecha de G, que pertenecen a toda clave. Son 33: los de las relaciones sin dependencias no triviales (las siete M:N y `Servicio`), los de las claves compuestas de `Compone`, `Distribuye`, `CubreDieta` y `Consume`, `id_menu_validado`, e `id_valoracion`, `id_dieta`, `id_nutricionista`, `id_paciente` e `id_sala`.

N no basta: $N^+ \neq U$. Quedan fuera diez clases de atributos que se determinan entre sí y que ningún atributo de N alcanza: `id_item` y `nombre_item`; `id_menu` e `id_menu_origen`; `id_revalorizacion` e `id_valoracion_generada`; el `id` y el `nombre` de cada uno de los seis nomencladores con nombre único; e `id_resultado`, `nombre_resultado` y `valor_resultado`. Una clave necesita un representante de cada clase, y basta con uno: aquí se elige el identificador. Resulta:

```
K = { id_nutricionista_autorizado, id_dieta_autorizada,
      id_paciente_inscrito,        id_dieta_inscrita,
      id_dieta_restringida,        id_restriccion_dieta,
      id_restriccion_excluyente,   id_grupo_excluido,
      id_menu_incluyente,          id_item_incluido,
      id_menu_cubierto,            id_grupo_cubierto_menu,
      id_menu_asignado,            id_paciente_asignado,
      id_menu_servido,  id_paciente_servido,  fecha_servicio,
      id_menu_consumo,  id_paciente_consumo,  fecha_servicio_consumo,
      id_item_consumido,
      id_menu_validado,
      id_plato_compuesto,          id_alimento_componente,
      id_menu_distribuido,         id_tipo_distribuido,
      id_dieta_cubierta,           id_grupo_cubierto,
      id_item, id_menu, id_valoracion, id_revalorizacion,
      id_dieta, id_nutricionista, id_paciente, id_sala,
      id_grupo, id_tipo, id_nivel, id_programa,
      id_restriccion, id_especialidad, id_resultado }
```

$K^+ = U$: los identificadores de entidad arrastran todos sus atributos, y los de interrelación ya

están en K. Y K es **mínima**, porque ninguno de sus atributos se deriva de los demás: quitar cualquiera lo deja fuera del cierre. Es, por tanto, una clave candidata de U. No es la única: elegir otro representante de alguna clase (por ejemplo, `nombre_item` en lugar de `id_item`) da otra, y para el lema de Ullman basta con una.

Obsérvese que la clave es un **conjunto de atributos**, no un conjunto de pares: los pares son las claves de cada interrelación por separado, no la del esquema universal.

**Ningún $R_i$ contiene K**, así que se añade `R_K(K)`. Ese esquema es *todo clave* y, al no proyectarse sobre él ninguna dependencia no trivial, está en BCNF. Pero reúne hechos independientes entre sí: qué nutricionista está autorizado en qué dieta nada tiene que ver con qué ítem compone qué plato.

**Las dependencias del dominio no son solo funcionales.** Gracias a los nombres de rol, U se parte en **28 bloques** de atributos que no se solapan: los 20 de los esquemas de §5.3.1 y los de las ocho relaciones sin dependencias no triviales (las siete M:N y `Servicio`). Los atributos de una interrelación 1:N, como `id_sala_paciente`, quedan en el bloque de su entidad. Cada bloque registra un hecho independiente de los demás: qué nutricionista está autorizado en qué dieta no tiene nada que ver con qué ítem incluye qué menú, ni con qué ingredientes tiene un plato. Esa independencia pertenece a la semántica del dominio y no se deja expresar con dependencias funcionales, así que se declara aparte, como un conjunto **M** de **dependencias multivaluadas**: por cada bloque X, se cumple $\emptyset \twoheadrightarrow X$. Toda dependencia de G cae dentro de un solo bloque, de modo que M es consistente con F y no implica dependencias funcionales nuevas. Dentro de `R_K`, cada una se proyecta sobre $K \cap X$. Las dependencias del dominio son, entonces, $\mathbf{D} = \mathbf{F} \cup \mathbf{M}$. La síntesis trabaja solo con F; M interviene únicamente aquí y, como no aporta ninguna dependencia funcional, la PPDF no se ve afectada.

**Descomposición de `R_K`.** `R_K` está en BCNF, pero no en 4FN respecto de M: dentro de él se cumple $\emptyset \twoheadrightarrow$ {`id_nutricionista_autorizado`, `id_dieta_autorizada`}, y una análoga por cada bloque. Por el teorema de Fagin, una dependencia multivaluada no trivial permite descomponer sin pérdida de información, y aplicándolo sucesivamente `R_K` se separa en un esquema por cada hecho independiente:

| Esquema | Atributos |
|---|---|
| `Autoriza` | id_nutricionista_autorizado, id_dieta_autorizada |
| `Inscribe` | id_paciente_inscrito, id_dieta_inscrita |
| `Restringe` | id_dieta_restringida, id_restriccion_dieta |
| `Excluye` | id_restriccion_excluyente, id_grupo_excluido |
| `Incluye` | id_menu_incluyente, id_item_incluido |
| `CubreMenú` | id_menu_cubierto, id_grupo_cubierto_menu |
| `Asignación` | id_menu_asignado, id_paciente_asignado |
| `Servicio` | id_menu_servido, id_paciente_servido, fecha_servicio |
| `Consume` (contenido en el esquema de §5.3.1, se elimina) | id_menu_consumo, id_paciente_consumo, fecha_servicio_consumo, id_item_consumido |

Los nombres de rol son los que hacen que esta partición funcione: la terna de `Servicio` y la cuaterna de `Consume` son **conjuntos de atributos disjuntos**, de modo que ninguna contiene a

la otra y las dos dependencias multivaluadas son independientes. Si el consumo reutilizara los atributos del servicio, la terna quedaría contenida en la cuaterna, el último paso del algoritmo eliminaría `Servicio` y el modelo perdería los servicios sin consumos registrados, que la §4.5 admite expresamente.

Los demás fragmentos de `R_K` son las proyecciones `K ∩ X` del resto de los bloques: los pares de `Compone`, `Distribuye` y `CubreDieta`, la cuaterna de `Consume`, el menú validado y los identificadores de entidad. Como M también los separa, cada uno queda como un esquema propio, **contenido** en uno de los ya obtenidos. Se eliminan, igual que en el último paso de la síntesis: si S ⊆ T, entonces `π_T(r) ⋈ π_S(r) = π_T(r)`, y quitar S no cambia el join. Los ocho esquemas que quedan son todo clave y están en BCNF, y con ellos `R_K` alcanza la 4FN.

**La descomposición final** es τ = ρ ∪ {los 8 de este apartado} = **28 esquemas**, donde ρ son los 20 esquemas de §5.3.1. Son exactamente las 28 tablas del esquema implementado.

---

## 5.4 Propiedades de la descomposición

### 5.4.1 Preservación de dependencias funcionales (PPDF)

La síntesis **garantiza la PPDF por construcción**. Cada dependencia `X → A` de G dio origen a un esquema $R_i$ con $X \cup \{A\} \subseteq U_i$, de modo que $X \to A \in \Pi_{U_i}(G^+)$. Por tanto:

$$G \subseteq \cup_i \Pi_{R_i}(G^+) \quad \text{y, como } G \text{ es un cubrimiento de } F,$$
$$F \equiv \cup_i \Pi_{R_i}(G^+)$$

Ninguna dependencia del dominio quedó repartida entre dos esquemas, así que toda comprobación de integridad asociada a una dependencia funcional se hace localmente, sin join. El paso posterior lo conserva: la descomposición de `R_K` solo afecta a atributos sobre los que no se proyecta ninguna dependencia no trivial.

### 5.4.2 Join sin pérdida (PLJ)

La síntesis **no** garantiza por sí sola la PLJ: hay que añadir un esquema que contenga una clave candidata del esquema universal. Es el resultado que justifica el paso de §5.3.3, y el que se conoce como lema de Ullman:

> Si una descomposición ρ de R(U, F) preserva las dependencias y alguno de sus esquemas contiene una clave candidata de R, entonces ρ tiene la propiedad del join sin pérdida.

Las dos condiciones se cumplen: la PPDF quedó demostrada en §5.4.1, y la clave K calculada en §5.3.3 está contenida en `R_K`. La descomposición σ = ρ ∪ {R_K} tiene, por tanto, la PLJ.

El último paso descompone `R_K` por las dependencias multivaluadas de M. No forma parte del algoritmo de síntesis, que trabaja solo con dependencias funcionales, sino que es un **paso posterior de descomposición a 4FN**, y por eso sus garantías se demuestran en lugar de invocarse: el teorema de Fagin establece que descomponer según una dependencia multivaluada no trivial es sin pérdida, y la composición de dos descomposiciones sin pérdida lo es también; quitar después los fragmentos contenidos en otro esquema no cambia el join (§5.3.3). La propiedad se conserva, por tanto, en la descomposición final τ de 28 esquemas, **respecto de D = F ∪ M**: con F sola, partir `R_K` sí perdería información, y por eso M se declara como parte de las dependencias del dominio. La PPDF no se ve afectada, porque sobre los atributos que ese paso separa no se proyecta ninguna dependencia funcional.

**Alcance del resultado.** Como los bloques no comparten atributos, el join natural de τ es, formalmente, su producto: la PLJ se demuestra en el marco de la relación universal con nombres de rol. Las combinaciones que hacen las consultas no son ese join, sino las que siguen las claves foráneas, que son dependencias de inclusión (§4.1.2) y quedan fuera de la teoría de dependencias funcionales y multivaluadas. Ese es el precio de los nombres de rol, y es también lo que evita que el algoritmo deduzca dependencias falsas (§5.1.2).

| Grupo | Verificación |
|---|---|

### 5.4.3 Verificación adicional: BCNF

La BCNF no forma parte del criterio de corrección, pero se comprueba esquema por esquema porque este diseño la alcanza. Un esquema está en BCNF si en toda dependencia funcional no trivial de $\Pi_{R_i}(G^+)$ la parte izquierda es superclave.

| Grupo | Verificación |
|---|---|
| Los 8 esquemas todo clave de §5.3.3 | Sobre ellos no se proyecta ninguna dependencia no trivial. **BCNF** |
| `Compone`, `Distribuye`, `CubreDieta`, `Consume` | Su único determinante es la clave compuesta completa, como mostró §5.2.2 al descartar los atributos extraños. **BCNF** |
| Los 6 nomencladores de nombre único y `ResultadoNutricional` | Dos o tres determinantes, y todos son claves candidatas. **BCNF** |
| `Sala`, `Dieta`, `Nutricionista`, `Paciente`, `Valoración` | Único determinante, la clave primaria. **BCNF** |
| `ItemAlimentario` | Dos determinantes, `id_item` y `nombre_item`, los dos claves candidatas. **BCNF** |
| `Menú` y `Validación` | En `Menú`, `id_menu` y, sobre las filas donde está definida, `id_menu_origen`. En `Validación`, solo `id_menu_validado`. **BCNF** |
| `Revalorización` | `id_revalorizacion` y, sobre las filas donde está definida, `id_valoracion_generada`. **BCNF** |

**Los 28 esquemas están en BCNF**, que es una condición más fuerte que la 3FN exigida por el criterio. El diseño anterior tenía una excepción, `Revalorización` en 2FN por la dependencia transitiva que introducía el paciente solicitante; al dejar de almacenarse, la excepción desaparece.

---

## 5.5 Valoración final del diseño

### 5.5.1 Completitud respecto a los requerimientos

El modelo cubre de forma integral los requerimientos funcionales, no funcionales e informacionales:

- Se gestiona el ciclo completo del ítem alimentario: ingreso, clasificación, composición de los platos y ficha nutricional ampliada.
- Se modelan las dietas con su cobertura y sus restricciones, y los menús con su parametrización de generación, su dictamen y, cuando procede, el menú al que reemplazan.
- Se registran las asignaciones, cada servicio de un menú a un paciente, los consumos por ítem en cada servicio, las valoraciones y las revalorizaciones.
- El esquema ofrece trayectoria de navegación directa para las seis consultas informacionales y para las dos recurrentes:

| Consulta | Trayectoria en el esquema |
|---|---|
| Consulta 1 — Menús automáticos por dieta | `Dieta` ← `Menú` (filtrado por `EsAutomático`) → `Crea` → `Nutricionista` (creador) · `FechaCreación` · `CantidadTotalAlimentos` · `Distribuye` · `CubreMenú` |
| Consulta 2 — Alimentos más usados en menús finales | `Dieta` ← `Menú` (con validación aprobada) → `Incluye` → `ItemAlimentario`: un `Alimento` cuenta él mismo, y un `Plato` → `Compone` → sus `Alimento`; cada alimento → `NivelCalórico`, `GrupoNutricional` |
| Consulta 3 — Menús validados por un revisor | `Nutricionista` (revisor) ← `Validación` (`FechaValidación`, `Aprobado`, `Observaciones`) → `Menú` → `Dieta`; `Menú` → `Nutricionista` (creador); si el dictamen es un rechazo, `Validación` → `Origina` → `Menú` de reemplazo |
| Consulta 4 — Desempeño ante un menú | `Menú` ← `Asignación` → `Servicio` → `Consume` (`Aceptación`) → `ItemAlimentario` → `NivelCalórico` |
| Consulta 5 — Comparación entre dietas | `Dieta` ← `Menú` → `Incluye` → `ItemAlimentario` → `GrupoNutricional`, `NivelCalórico`; requeridos y restringidos: `CubreDieta.Tipo`; excluidos: `Restringe` → `Excluye`; parámetros: `Distribuye`, `CantidadTotalAlimentos` |
| Consulta 6 — Correlación nivel calórico / resultado | (A) `Dieta` ← `Menú` → `Incluye` → `ItemAlimentario` → `NivelCalórico`; `Menú` ← `Asignación` ← `Valoración` → `ResultadoNutricional.Valor` · (B) `Consume` → `ItemAlimentario` → `Nutricionista` descriptor; `Consume` → `Servicio` → `Asignación` → `Menú` → `Dieta` · (C) `Revalorización` → `Genera` → `Valoración` generada → `ResultadoNutricional.Valor`; `Revalorización` → `Revalora` → `Valoración` revisada → `Asignación` → `Paciente` → `Sala`, y `Asignación` → `Menú` → `Dieta`; promedio por (sala, dieta) de las demás valoraciones |
| R1 — Disponibilidad calórica | `Dieta` → `Restringe` → `Excluye` y `Dieta` → `CubreDieta` (grupos prohibidos); `ItemAlimentario` → `PerteneceGrupo` → `GrupoNutricional` (compatibilidad) y `ItemAlimentario` → `NivelCalórico`; `Dieta` ← `Menú` final → `Incluye` → `ItemAlimentario` → `NivelCalórico` |
| R2 — Valoraciones más recientes | `Paciente` ← `Asignación` ← `Valoración` → `ResultadoNutricional`, ordenado por `FechaValoración` |
| RF-07 — Menús de la misma dieta | `Nutricionista` → `Autoriza` → `Dieta` ← `Menú` |

Los cálculos —tasas, promedios, correlaciones, el corte del top-10 y los conteos de disponibilidad— no están almacenados: se derivan en la consulta. RNF-01 se satisface sirviendo R1 y R2 desde el caché descrito en §2.2, sin que PostgreSQL deje de ser la fuente de verdad.

## 5.6 Flexibilidad y extensibilidad

El diseño presenta una estructura modular que facilita su evolución ante cambios del dominio nutricional. La independencia entre entidades base (ítems alimentarios y grupos nutricionales) y las estructuras operativas (menús, dietas, consumos) permite incorporar nuevos tipos de ítem, criterios dietéticos o modalidades de valoración sin alterar las tablas existentes. La separación entre los parámetros de generación de un menú (`Distribuye`, `CubreMenú`) y su composición efectiva (`Incluye`) garantiza que las reglas de negocio puedan ajustarse sin reestructurar el esquema. El uso de nomencladores abiertos para `GrupoNutricional` y `ProgramaDeAtención` permite ampliar los catálogos sin cambios estructurales. La externalización de la ficha nutricional a MongoDB ofrece flexibilidad para extender los datos de nutrientes sin migraciones relacionales.

No quedan decisiones de modelado abiertas. Las cuatro que el diseño podía dejar en suspenso están fijadas: la `Cantidad` de la composición se expresa en gramos (RD-05); la `Proporción` de la distribución es una fracción en el intervalo (0, 1] y la suma de las de un menú es 1 (RD-11, RS-08); la `Edad` del paciente es un número entero de años mayor o igual que cero (RD-06, RD-09); y la correspondencia entre las restricciones de una dieta y los alérgenos de sus ítems queda deliberadamente fuera del modelo relacional (§4.5).

Esa última merece una palabra, porque es el punto de extensión más claro del diseño. La compatibilidad alimentaria se resuelve hoy en dos niveles: por grupo nutricional dentro de PostgreSQL, y por alérgeno en la capa de aplicación, que contrasta la ficha de cada ítem contra las restricciones de la dieta. Lo que el modelo relacional no registra es **qué alérgeno excluye cada restricción** —que *sin gluten* excluye *gluten*—, una correspondencia que queda en la configuración de la aplicación. Llevarla al esquema, con un nomenclador de alérgenos y una relación hacia `RestricciónAlimentaria`, permitiría comprobar la compatibilidad por alérgeno sin salir del modelo. Es una extensión aditiva: no obliga a reestructurar ninguna tabla existente, porque el mecanismo por grupo seguiría funcionando igual.

# 6. Conclusiones

El informe recorre las cuatro etapas del diseño: análisis de los requerimientos funcionales, no funcionales e informacionales; modelo conceptual (MERX consolidado); especificación de 103 restricciones de integridad, clasificadas por su origen y con el mecanismo que las implementa (§4); y valoración formal de la corrección.

El esquema relacional se obtuvo así: del modelo conceptual se extrajo el conjunto F de dependencias funcionales, se calculó su cubrimiento minimal G y se le aplicó el algoritmo de síntesis, que da 20 esquemas más el esquema de la llave. Ese último se descompuso a 4FN según las dependencias multivaluadas del dominio, y el resultado son **28 esquemas**. La descomposición cumple las tres condiciones del criterio de corrección: cada esquema alcanza al menos la 3FN, se preservan todas las dependencias funcionales del dominio —garantizado por construcción del algoritmo— y el join natural no introduce tuplas espurias: la descomposición que incluye el esquema de la llave es sin pérdida por el lema de Ullman, y partir ese esquema según las dependencias multivaluadas del dominio también lo es, por el teorema de Fagin. La verificación adicional muestra que los 28 esquemas alcanzan además la Forma Normal de Boyce-Codd, condición más fuerte que la exigida.

La base de datos propuesta es, por tanto, estructuralmente correcta y está preparada para su implementación.